



US 20250287020A1

(19) **United States**

(12) **Patent Application Publication**
SKUPIN et al.

(10) **Pub. No.: US 2025/0287020 A1**

(43) **Pub. Date: Sep. 11, 2025**

(54) **EFFICIENT SUB-PICTURE EXTRACTION**

(30) **Foreign Application Priority Data**

(71) Applicant: **FRAUNHOFER-GESELLSCHAFT
ZUR FÖRDERUNG DER
ANGEWANDTEN FORSCHUNG
E.V., München (DE)**

Jan. 25, 2018 (EP) 18153516.2

Publication Classification

(72) Inventors: **Robert Philipp SKUPIN**, Berlin (DE);
Cornelius HELIGE, Berlin (DE);
Valeri GEORGE, Berlin (DE);
Thomas SCHIERL, Berlin (DE); **Yago
SANCHEZ DE LA FUENTE**, Berlin
(DE); **Karsten SÜHRING**, Berlin
(DE); **Thomas WIEGAND**, Berlin
(DE)

(51) **Int. Cl.**

H04N 19/17 (2014.01)
H04L 65/70 (2022.01)
H04L 65/756 (2022.01)
H04N 19/119 (2014.01)
H04N 19/174 (2014.01)
H04N 19/46 (2014.01)
H04N 19/70 (2014.01)
H04N 19/91 (2014.01)

(52) **U.S. Cl.**

CPC **H04N 19/17** (2014.11); **H04L 65/70**
(2022.05); **H04L 65/756** (2022.05); **H04N**
19/119 (2014.11); **H04N 19/174** (2014.11);
H04N 19/46 (2014.11); **H04N 19/70**
(2014.11); **H04N 19/91** (2014.11)

(21) Appl. No.: **19/212,213**

(22) Filed: **May 19, 2025**

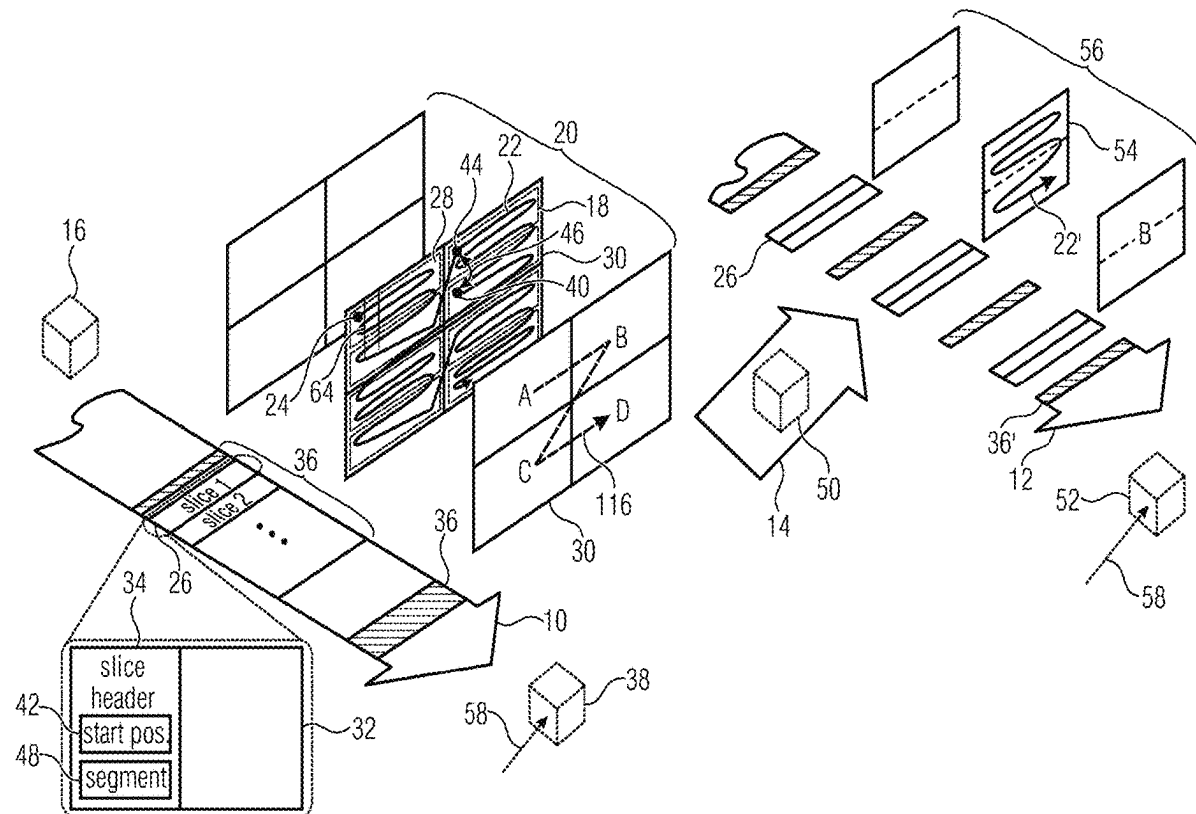
Related U.S. Application Data

(63) Continuation of application No. 18/493,002, filed on Oct. 24, 2023, now Pat. No. 12,348,736, which is a continuation of application No. 17/568,512, filed on Jan. 4, 2022, now Pat. No. 11,838,504, which is a continuation of application No. 16/930,231, filed on Jul. 15, 2020, now Pat. No. 11,245,898, which is a continuation of application No. PCT/EP2019/051394, filed on Jan. 21, 2019.

(57)

ABSTRACT

Sub-picture extraction is rendered less complex by providing each slice with start position information which indicates a starting position from which onwards the picture is, along a coding path, encoded into the respective slice, relative to an onset position at which the coding path starts traversing a segment the starting position is located in. Alternatively or additionally, sub-picture extraction processes are treated like choosing one layer out of a multi-layer datastream.



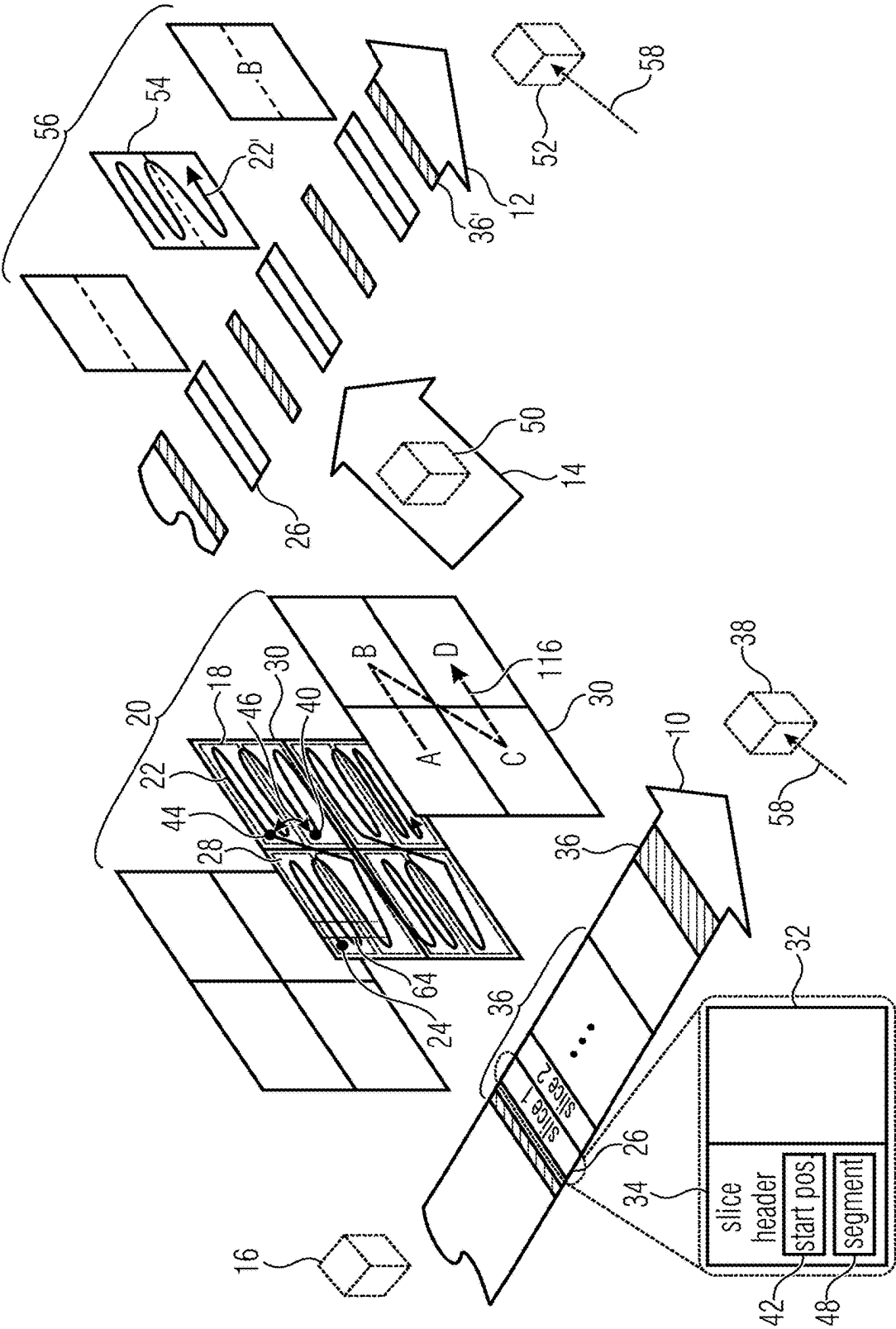


Fig. 1

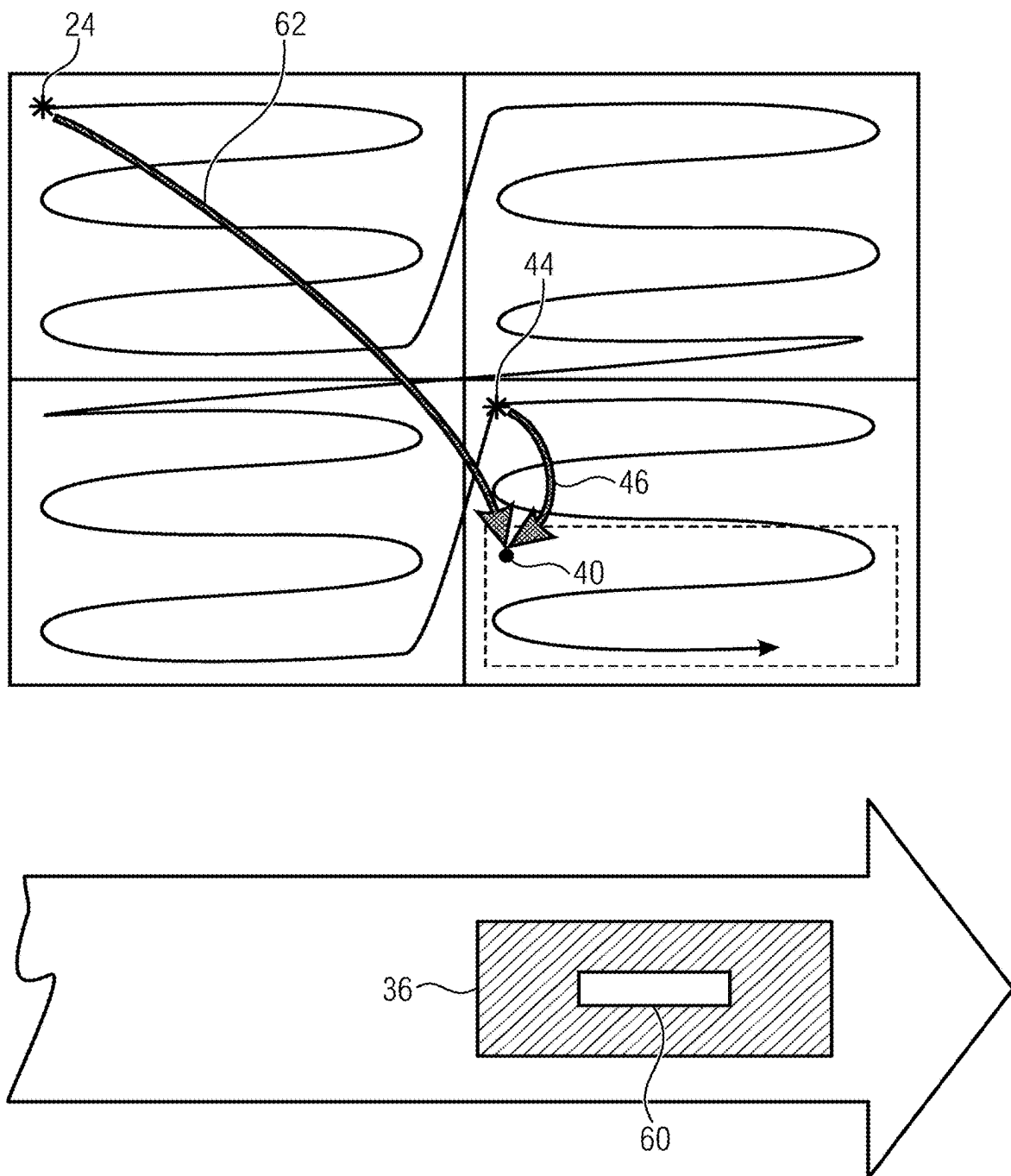


Fig. 2

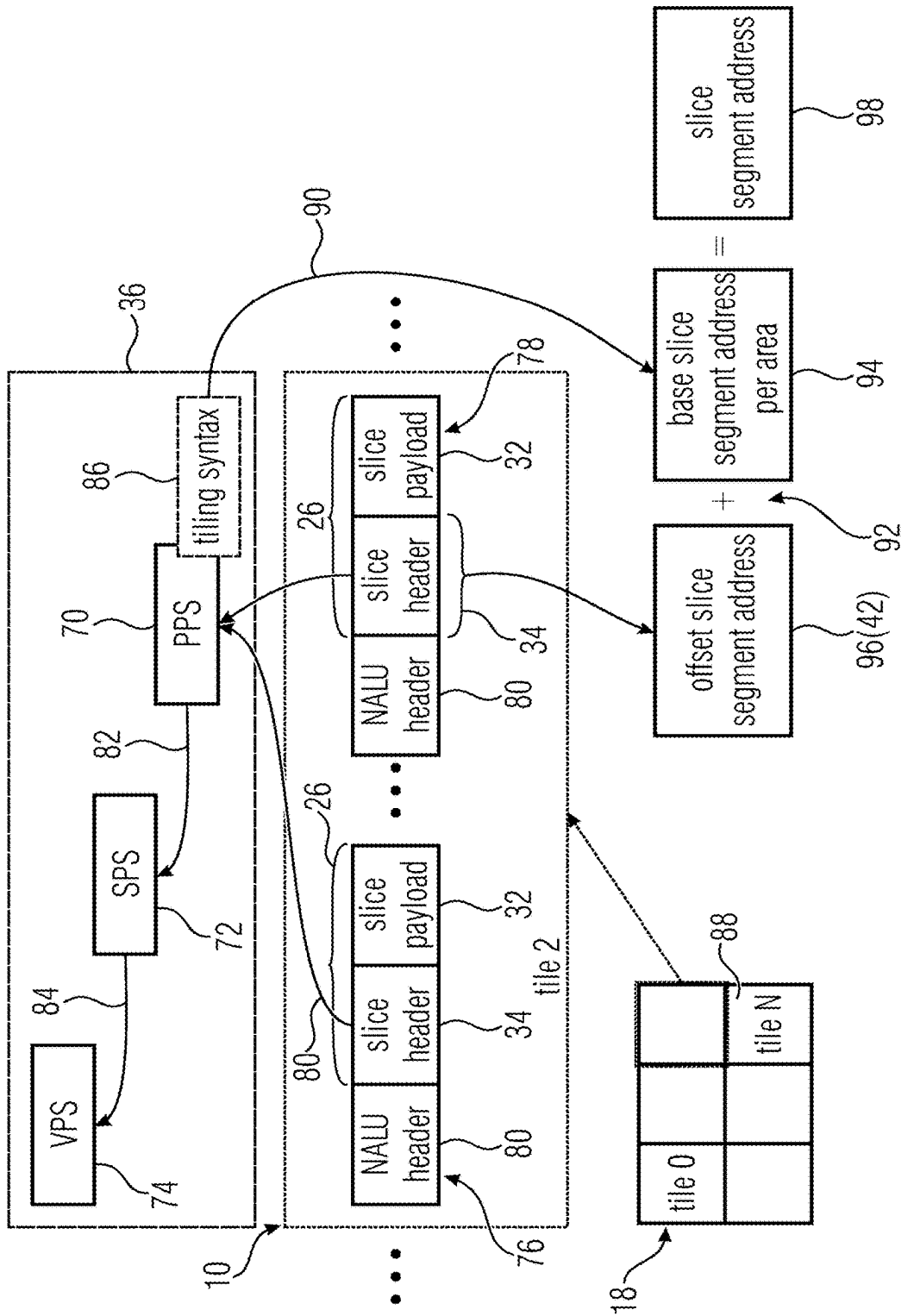


Fig. 3

70

	Descriptor
pic_parameter_set_rbsp() {	
pps_pic_parameter_set_id	ue(v)
pps_seq_parameter_set_id	ue(v)
dependent_slice_segments_enabled_flag	u(1)
[...]	
tiles_enabled_flag	u(1)
entropy_coding_sync_enabled_flag	u(1)
if(tiles_enabled) {	
102 num_tile_columns_minus1	ue(v)
104 num_tile_rows_minus1	ue(v)
106 uniform_spacing_flag	u(1)
if(!uniform_spacing_flag){	
for(i = 0; i < num_tile_columns_minus1; i++)	
108 column_width_minus1[i]	ue(v)
for(i = 0; i < num_tile_rows_minus1; i++)	
110 row_height_minus1[i]	ue(v)
}	
loop_filter_across_tiles_enabled_flag	u(1)
112 slice_segment_base_addr_per_tile_enable_flag	u(1)
if (slice_segment_base_addr_per_tile_enable_flag){	
for(i = 0; i < ((num_tile_columns_minus1 + 1) * (num_tile_rows_minus1 + 1) - 1); i++)	
114 tile_id_in_pps[i]	u(8)
}	
}	
}	

100

Fig. 4

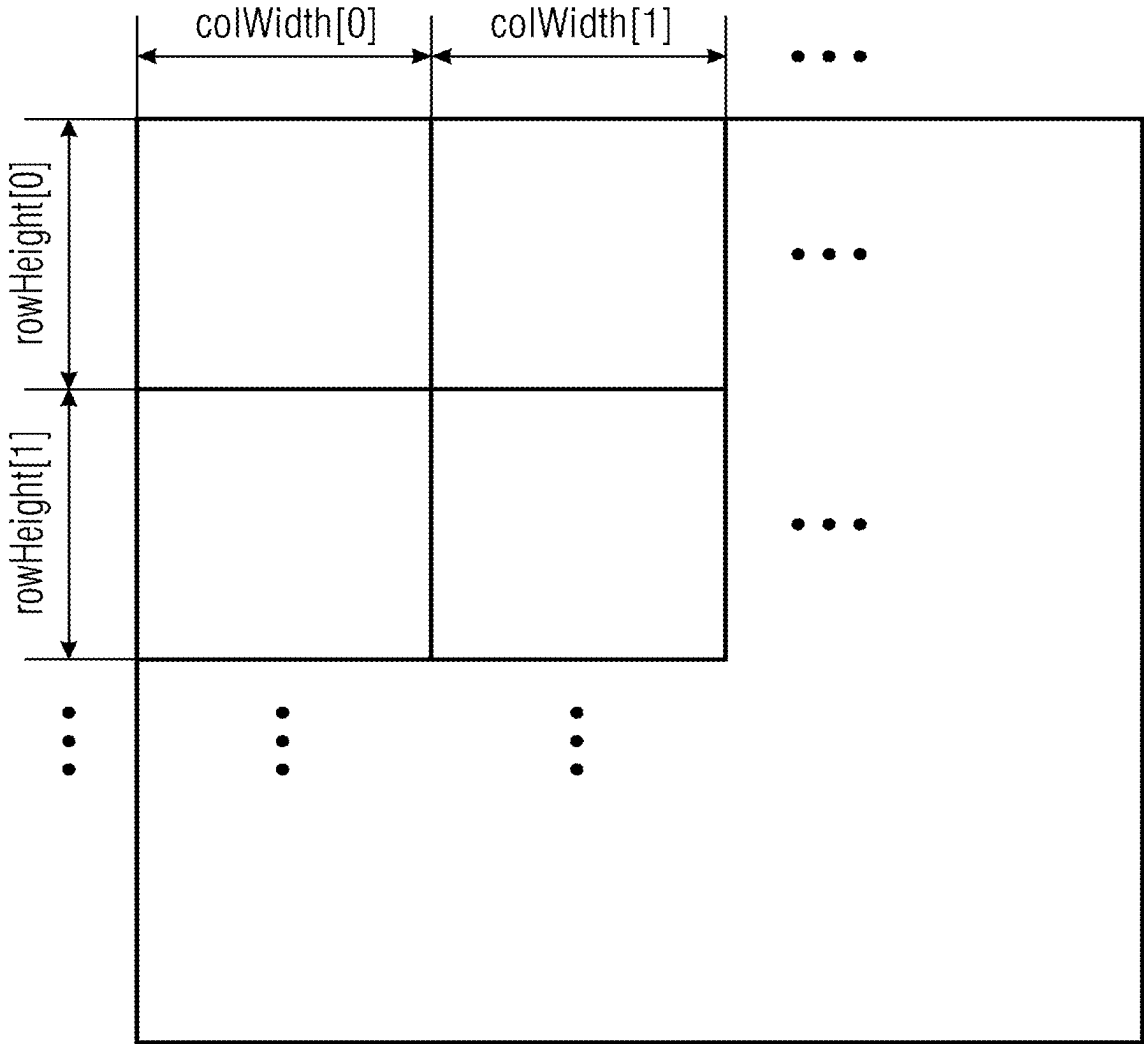


Fig. 5

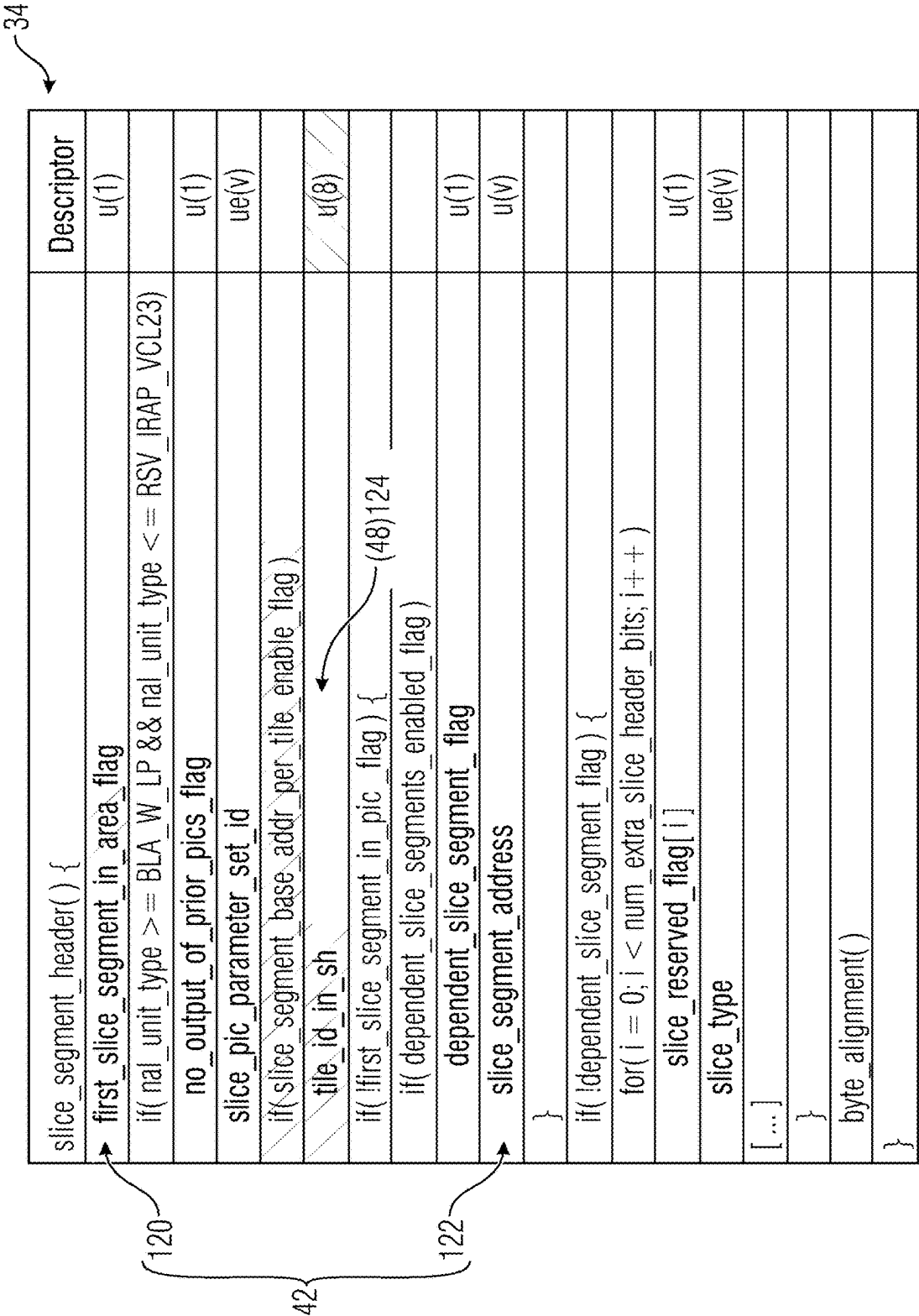


Fig. 6

	Descriptor
tiling_syntax() {	
num_tile_columns_minus1	ue(v)
num_tile_rows_minus1	ue(v)
uniform_spacing_flag	u(1)
if (!uniform_spacing_flag) {	
for(i = 0; i < num_tile_columns_minus1; i++)	
column_width_minus1[i]	ue(v)
for(i = 0; i < num_tile_rows_minus1; i++)	
row_height_minus1[i]	ue(v)
}	
loop_filter_across_tiles_enabled_flag	u(1)
slice_segment_base_addr_per_area_enable_flag	u(1)
if(slice_segment_base_addr_per_area_enable_flag) {	
k=0; NumTilesInArea[0,255] = 0	
for(i = 0; i < ((num_tile_columns_minus1 + 1) * (num_tile_rows_minus1 + 1) - 1); i++)	
area_id_in_ps[i]	u(8)
NumTilesInArea[area_id_in_ps[i]] ++	
if(i != 0 && (NumTilesInArea[area_id_in_ps[i]] == 1))	
slice_segment_base_address[k++]	ue(v)
}	
num_areas = k + 1	
}	

130

Fig. 7

seq_parameter_set_rbsp() {	Descriptor
sps_video_parameter_set_id	u(4)
sps_max_sub_layers_minus1	u(3)
[...]	
tiles_enabled_in_sps_flag	u(1)
if(tiles_enabled_in_sps_flag) {	
tiling_syntax()	
[...]	
rbsp_trailing_bits()	
}	

Fig. 8

pic_parameter_set_rbsp() {	Descriptor
pps_pic_parameter_set_id	ue(v)
pps_seq_parameter_set_id	ue(v)
dependent_slice_segments_enabled_flag	u(1)
[...]	
tiles_enabled_in_pps_flag	u(1)
entropy_coding_sync_enabled_flag	u(1)
if(tiles_enabled_in_pps_flag) {	
tiling_syntax()	
[...]	
rbsp_trailing_bits()	
}	

Fig. 9

Descriptor	
	slice_segment_header() {
u(1)	first_slice_segment_in_area_flag
	if(nal_unit_type >= BLA_W_LP && nal_unit_type <= RSV_IRAP_VCL23)
u(1)	no_output_of_prior_pics_flag
ue(v)	slice_pic_parameter_set_id
	if(slice_segment_base_addr_per_area_enable_flag)
u(8)	area_id_in_sh
	if(!first_slice_segment_in_pic_flag) {
	if(dependent_slice_segments_enabled_flag)
u(1)	dependent_slice_segment_flag
u(v)	slice_segment_address
	}
	if(!dependent_slice_segment_flag) {
	for(i = 0; i < num_extra_slice_header_bits; i++)
u(1)	slice_reserved_flag[i]
ue(v)	slice_type
	[...]
	}
	byte_alignment()
	}

Fig. 10

	seg _parameter_set_rbsp() {	Descriptor
	sps_video_parameter_set_id	u(4)
	sps_max_sub_layers_minus1	u(3)
	[...]	
	tiles_enabled_in_sps_flag	u(1)
	if(tiles_enabled_in_sps_flag) {	
140	→ tiling_syntax()	
148	→ num_output_tile_formats	ue(v)
	for(i = 0; i < num_output_tile_formats - 1; i++) {	
142	→ pic_width_in_luma_samples[i]	ue(v)
144	→ pic_height_in_luma_samples[i]	ue(v)
146	→ profile_tier_level()[i]	
	}	
150	→ num_output_tile_sets	ue(v)
	for(i = 0; i < num_output_tile_sets - 1; i++) {	
154	→ num_tiles_in_output_tile_set[i]	ue(v)
152	→ output_tile_set_format_idx[i]	ue(v)
	for(j = 0; j < num_tiles_in_output_tile_set - 1; j++) {	
156	→ tile_id_in_sps[i][j]	ue(v)
	}	ue(v)
158	→ hrd_parameters()	
	}	
	[...]	
	rbp_trailing_bits()	
	}	

Fig. 11

EFFICIENT SUB-PICTURE EXTRACTION**CROSS-REFERENCES TO RELATED APPLICATIONS**

[0001] This application is a continuation of U.S. patent application Ser. No. 18/493,002, filed Oct. 24, 2023, pending, which is a continuation of U.S. patent application Ser. No. 17/568,512, filed Jan. 4, 2022, issued as U.S. Pat. No. 11,838,504 on Dec. 5, 2023, which is a continuation of U.S. patent application Ser. No. 16/930,231, filed Jul. 15, 2020, issued as U.S. Pat. No. 11,245,898 on Feb. 8, 2022, which is a continuation of PCT International Application No. PCT/EP2019/051394, filed Jan. 21, 2019, which claims priority of European Patent Application No. 18153516.2, filed Jan. 25, 2018, the entire contents of each of which are hereby incorporated by reference in this application.

[0002] The present application is concerned with concepts for efficient sub-picture extraction.

BACKGROUND OF THE INVENTION

[0003] Sub-picture extraction is a process where a datastream, which has encoded therein a video picture, is, without the need for re-encoding, tailored to a sub-picture area. HEVC DAM3 MCTS extraction, for instance, allows for an extraction of a sub-picture-specific datastream from an original full-picture datastream without the need for any re-encoding as far as motion compensation prediction and residual coding, for instance, is concerned, by sub-dividing the pictures constantly into tiles which are coded independently from each other, and grouping the tiles into tile sets with respect to which even the motion-compensated prediction is restricted not to cross tile set boundaries. This MCTS extraction, however, involves adjustment of each NAL unit carrying slice data in order to adjust the slice segment address of the carried slice header.

[0004] Accordingly, sub-picture extraction processes are known, but it would be favorable to have concepts at hand which decrease the complexity of the tasks involved.

SUMMARY

[0005] An embodiment may have a data stream having a picture encoded therein

[0006] wherein the picture is coded into the data stream along a coding path in units of slices into which the picture is partitioned,

[0007] wherein the picture is subdivided into segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded therein, wherein the picture has encoded therein the segments without coding interdependencies,

[0008] wherein each slice includes

[0009] start position information indicating a starting position from which onwards the picture is, along the coding path, encoded into the respective slice, wherein the starting position information indicates the starting position relative to an onset position at which the coding path starts traversing a segment the starting position is located in, and

[0010] segment information which indicates the segment the starting position is located in.

[0011] Another embodiment may have a decoder for decoding a picture from a data stream,

[0012] wherein the picture is coded into the data stream along a coding path in units of slices into which the picture is partitioned,

[0013] wherein the picture is subdivided into segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded therein, wherein the picture has encoded therein the segments without coding interdependencies,

[0014] wherein the decoder is configured to, for each slice,

[0015] decode, from the data stream, a start position information,

[0016] locate, using the start position information, a starting position from which onwards the picture is encoded into the respective slice, by using the start position information for locating the starting position relative to an onset position at which the coding path starts, along the coding path, traversing a segment the starting position is located in, and

[0017] decode, from the data stream, segment information which indicates the segment the starting position is located in.

[0018] Another embodiment may have a data stream having a picture encoded therein

[0019] wherein the picture is coded into the data stream along a coding path in units of slices into which the picture is partitioned,

[0020] wherein the picture is subdivided into segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded therein, wherein the picture has encoded therein the segments without coding interdependencies,

[0021] wherein each slice includes

[0022] a start position information indicating a starting position from which onwards the picture is encoded into the respective slice,

[0023] wherein the data stream includes a start position coding mode flag which is switchable between a first state and a second state, wherein,

[0024] if in the first state,

[0025] the start position information indicates the starting position relative to an onset position at which the coding path starts traversing a segment the starting position is located in, and each slice includes segment information indicating the segment the starting position is located in, and

[0026] if in the second state,

[0027] the start position information indicates the starting position relative to a picture onset position at which the coding path starts traversing the picture, with the slices being without the segment information.

[0028] Yet another embodiment may have a decoder for decoding a picture from a data stream,

[0029] wherein the picture is coded into the data stream along a coding path in units of slices into which the picture is partitioned,

[0030] wherein the picture is subdivided into segments which are traversed by the coding path sequentially,

segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded thereinto, wherein the picture is encoded into the segments without coding interdependencies,

- [0031] wherein the decoder is configured to, for a predetermined slice
- [0032] decode a start position information indicating a starting position from which onwards the picture is encoded into the predetermined slice,
- [0033] decode a start position coding mode flag which is switchable between a first state and a second state from the data stream,
- [0034] if in the first state,
- [0035] locate, using the start position information, the starting position relative to an onset position at which the coding path starts traversing a segment the starting position is located in, and decode, from the data stream, segment information indicating the segment the starting position is located in, and
- [0036] in the second state,
- [0037] locate, using the start position information, the starting position relative to a picture onset position at which the coding path starts traversing the picture, and determining the segment the starting position is located in on the basis of the starting position.
- [0038] Yet another embodiment may have a data stream having a picture encoded thereinto
- [0039] wherein the picture is coded into the data stream along a coding path in units of slices into which the picture is partitioned,
- [0040] wherein the picture is subdivided into segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded thereinto, wherein the picture has encoded thereinto the segments without coding interdependencies,
- [0041] wherein each slice includes segment information which indicates the segment the respective slice covers,
- [0042] wherein the data stream includes a parameter set indicating
- [0043] a size of the picture;
- [0044] an indication of a first decoder capability level that may be used for decoding the picture from the data stream;
- [0045] for at least one sub-picture decoding option,
- [0046] a reduced picture size corresponding to a segment cluster composed of a proper subset of the segments, and
- [0047] a second decoder capability level that may be used for decoding the segment cluster from an extracted version of the data stream which results from the data stream by stripping off slices from the data stream having segments encoded thereinto which are spatially offset to the segment cluster.
- [0048] Another embodiment may have a decoder configured to
- [0049] decode a parameter set from a data stream,
- [0050] derive a decoding option indication whether a picture decoding or sub-picture decoding is to be performed on the data stream,
- [0051] if the picture decoding is to be performed on the data stream,

[0052] derive from the parameter set a size of a picture and an indication of a first decoder capability level that may be used for decoding the picture from the data stream, and

- [0053] derive from the parameter set a first information on a subdivision of the picture into a first set of segments which are encoded into the data streams without coding interdependencies,
- [0054] decoding the picture from the data stream in units of slices into which the picture is partitioned along a first coding path, wherein the first coding path sequentially, segment by segment, traverses the picture and each slice has a fraction of a segment, or one or more segments completely, out of the first set of segments encoded thereinto,
- [0055] if the sub-picture decoding is to be performed on the data stream,
- [0056] derive from the parameter set a further picture size and an indication of a second decoder capability level that may be used for decoding a sub-picture of the further picture size from the data stream, and
- [0057] derive from the parameter set a second information on a subdivision of the sub-picture into a second set of segments which segments are encoded into the data streams without coding interdependencies and which second set is a subset of the first set of segments,
- [0058] decoding the sub-picture from the data stream in units of the slices into which the sub-picture is partitioned along a second coding path, wherein the second coding path sequentially, segment by segment, traverses the sub-picture and each slice has a fraction of a segment, or one or more segments completely, out of the second set of segments encoded thereinto,
- [0059] wherein the decoder is configured to, for each slice,
- [0060] decode, from the data stream, segment information which indicates a segment out of the first set of segments, in which a starting position is located from which onwards the respective slice is encoded.
- [0061] Another embodiment may have a method for decoding a picture from a data stream,
- [0062] wherein the picture is coded into the data stream along a coding path in units of slices into which the picture is partitioned,
- [0063] wherein the picture is subdivided into segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded thereinto, wherein the picture has encoded thereinto the segments without coding interdependencies,
- [0064] wherein the method may have, for each slice, the steps of:
- [0065] decoding, from the data stream, a start position information,
- [0066] locating, using the start position information, a starting position from which onwards the picture is encoded into the respective slice, by using the start position information for locating the starting position relative to an onset position at which the coding path starts, along the coding path, traversing a segment the starting position is located in, and

- [0067] decoding, from the data stream, segment information which indicates the segment the starting position is located in.
- [0068] Another embodiment may have a method for decoding a picture from a data stream,
- [0069] wherein the picture is coded into the data stream along a coding path in units of slices into which the picture is partitioned,
- [0070] wherein the picture is subdivided into segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded thereinto, wherein the picture is encoded into the segments without coding interdependencies,
- [0071] wherein the method may have, for a predetermined slice, the steps of:
- [0072] decoding a start position information indicating a starting position from which onwards the picture is encoded into the predetermined slice,
- [0073] decoding a start position coding mode flag which is switchable between a first state and a second state from the data stream,
- [0074] if in the first state,
- [0075] locating, using the start position information, the starting position relative to an onset position at which the coding path starts traversing a segment the starting position is located in, and decode, from the data stream, segment information indicating the segment the starting position is located in, and
- [0076] in the second state,
- [0077] locating, using the start position information, the starting position relative to a picture onset position at which the coding path starts traversing the picture, and determining the segment the starting position is located in on the basis of the starting position.
- [0078] According to another embodiment, a decoding method may have the steps of:
- [0079] decoding a parameter set from a data stream,
- [0080] deriving a decoding option indication whether a picture decoding or sub-picture decoding is to be performed on the data stream,
- [0081] if the picture decoding is to be performed on the data stream,
- [0082] deriving from the parameter set a size of a picture and an indication of a first decoder capability level that may be used for decoding the picture from the data stream, and
- [0083] deriving from the parameter set a first information on a subdivision of the picture into a first set of segments which are encoded into the data streams without coding interdependencies,
- [0084] decoding the picture from the data stream in units of slices into which the picture is partitioned along a first coding path, wherein the first coding path sequentially, segment by segment, traverses the picture and each slice has a fraction of a segment, or one or more segments completely, out of the first set of segments encoded thereinto,
- [0085] if the sub-picture decoding is to be performed on the data stream,
- [0086] deriving from the parameter set a further picture size and an indication of a second decoder capability level that may be used for decoding a sub-picture of the further picture size from the data stream, and
- [0087] deriving from the parameter set a second information on a subdivision of the sub-picture into a second set of segments which segments are encoded into the data streams without coding interdependencies and which second set is a subset of the first set of segments,
- [0088] decoding the sub-picture from the data stream in units of the slices into which the sub-picture is partitioned along a second coding path, wherein the second coding path sequentially, segment by segment, traverses the sub-picture and each slice has a fraction of a segment, or one or more segments completely, out of the second set of segments encoded thereinto,
- [0089] for each slice,
- [0090] decoding, from the data stream, segment information which indicates a segment out of the first set of segments, in which a starting position is located from which onwards the respective slice is encoded.
- [0091] Another embodiment may have an encoder for encoding a picture into a data stream,
- [0092] wherein the encoder is configured to encode the picture into the data stream in a manner so that the picture is coded along a coding path in units of slices into which the picture is partitioned,
- [0093] wherein the encoder is configured to encode the picture into the data stream in a manner so that the picture is subdivided into segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded thereinto,
- [0094] wherein the encoder is configured to encode the segments without coding interdependencies,
- [0095] wherein the encoder is configured to, for each slice,
- [0096] encode, into the data stream, a start position information, so that, using the start position information, a starting position from which onwards the picture is encoded into the respective slice, is locatable by using the start position information for locating the starting position relative to an onset position at which the coding path starts, along the coding path, traversing a segment the starting position is located in, and
- [0097] encode, into the data stream, segment information which indicates the segment the starting position is located in.
- [0098] Yet another embodiment may have an encoder for encoding a picture into a data stream,
- [0099] wherein the encoder is configured to encode the picture into the data stream in a manner so that the picture is coded along a coding path in units of slices into which the picture is partitioned,
- [0100] wherein the encoder is configured to encode the picture into the data stream in a manner so that the picture is subdivided into segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded thereinto,
- [0101] wherein the encoder is configured to encode the segments without coding interdependencies,
- [0102] wherein the encoder is configured to, for a predetermined slice
- [0103] encode a start position information indicating a starting position from which onwards the picture is encoded into the predetermined slice,

- [0104] encode a start position coding mode flag which is switchable between a first state and a second state from the data stream, in a manner so that
- [0105] if in the first state,
- [0106] the start position information locates the starting position relative to an onset position at which the coding path starts traversing a segment the starting position is located in, and the encoder is configured to encode, into the data stream, segment information indicating the segment the starting position is located in, and
- [0107] if in the second state,
- [0108] the start position information locates the starting position relative to a picture onset position at which the coding path starts traversing the picture, so that the segment the starting position is located is determinable in on the basis of the starting position.
- [0109] Another embodiment may have an encoder configured to
- [0110] encode a parameter set into a data stream, wherein
- [0111] the parameter set indicates,
- [0112] for a picture decoding to be performed on the data stream, a size of a picture and an indication of a first decoder capability level that may be used for decoding the picture from the data stream, and a first information on a subdivision of the picture into a first set of segments which are encoded into the data streams without coding interdependencies, wherein according to the picture decoding, the picture is decodable from the data stream in units of slices into which the picture is partitioned along a first coding path, wherein the first coding path sequentially, segment by segment, traverses the picture and each slice has a fraction of a segment, or one or more segments completely, out of the first set of segments encoded thereinto,
- [0113] for a sub-picture decoding to be performed on the data stream, a further picture size and an indication of a second decoder capability level that may be used for decoding a sub-picture of the further picture size from the data stream, and a second information on a subdivision of the sub-picture into a second set of segments which segments are encoded into the data streams without coding interdependencies and which second set is a subset of the first set of segments, wherein according to the sub-picture decoding, the sub-picture is decodable from the data stream in units of the slices into which the sub-picture is partitioned along a second coding path, wherein the second coding path sequentially, segment by segment, traverses the sub-picture and each slice has a fraction of a segment, or one or more segments completely, out of the second set of segments encoded thereinto,
- [0114] encode, for each slice,
- [0115] into the data stream, segment information which indicates a segment out of the first set of segments, in which a starting position is located from which onwards the respective slice is encoded.
- [0116] Another embodiment may have a method for encoding a picture into a data stream,
- [0117] wherein the method may have the step of encoding the picture into the data stream in a manner so that the picture is coded along a coding path in units of slices into which the picture is partitioned,
- [0118] wherein the picture is encoded into the data stream in a manner so that the picture is subdivided into segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded thereinto,
- [0119] wherein the segments are encoded without coding interdependencies,
- [0120] wherein the method may have, for each slice, the steps of:
- [0121] encoding, into the data stream, a start position information, so that, using the start position information, a starting position from which onwards the picture is encoded into the respective slice, is locatable by using the start position information for locating the starting position relative to an onset position at which the coding path starts, along the coding path, traversing a segment the starting position is located in, and
- [0122] encoding, into the data stream, segment information which indicates the segment the starting position is located in.
- [0123] Yet another embodiment may have a method for encoding a picture into a data stream,
- [0124] which method may have the step of encoding the picture into the data stream in a manner so that the picture is coded along a coding path in units of slices into which the picture is partitioned,
- [0125] wherein the picture is encoded into the data stream in a manner so that the picture is subdivided into segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded thereinto,
- [0126] wherein the segments are encoded without coding interdependencies,
- [0127] wherein the method may have, for a predetermined slice, the steps of:
- [0128] encoding a start position information indicating a starting position from which onwards the picture is encoded into the predetermined slice,
- [0129] encoding a start position coding mode flag which is switchable between a first state and a second state from the data stream, in a manner so that
- [0130] if in the first state,
- [0131] the start position information locates the starting position relative to an onset position at which the coding path starts traversing a segment the starting position is located in, and the method may have the step of encoding, into the data stream, segment information indicating the segment the starting position is located in, and
- [0132] if in the second state,
- [0133] the start position information locates the starting position relative to a picture onset position at which the coding path starts traversing the picture, so that the segment the starting position is located is determinable in on the basis of the starting position.
- [0134] According to another embodiment, an encoding method may have the step of:
- [0135] encoding a parameter set into a data stream, wherein
- [0136] the parameter set indicates,
- [0137] for a picture decoding to be performed on the data stream, a size of a picture and an indication of a

first decoder capability level that may be used for decoding the picture from the data stream, and a first information on a subdivision of the picture into a first set of segments which are encoded into the data streams without coding interdependencies, wherein according to the picture decoding, the picture is decodable from the data stream in units of slices into which the picture is partitioned along a first coding path, wherein the first coding path sequentially, segment by segment, traverses the picture and each slice has a fraction of a segment, or one or more segments completely, out of the first set of segments encoded therein,

[0138] for a sub-picture decoding to be performed on the data stream, a further picture size and an indication of a second decoder capability level that may be used for decoding a sub-picture of the further picture size from the data stream, and a second information on a subdivision of the sub-picture into a second set of segments which segments are encoded into the data streams without coding interdependencies and which second set is a subset of the first set of segments, wherein according to the sub-picture decoding, the sub-picture is decodable from the data stream in units of the slices into which the sub-picture is partitioned along a second coding path, wherein the second coding path sequentially, segment by segment, traverses the sub-picture and each slice has a fraction of a segment, or one or more segments completely, out of the second set of segments encoded therein,

[0139] encoding, for each slice,

[0140] into the data stream, segment information which indicates a segment out of the first set of segments, in which a starting position is located from which onwards the respective slice is encoded.

[0141] According to another embodiment, a non-transitory digital storage medium may have a computer program stored thereon to perform the inventive methods, when said computer program is run by a computer.

[0142] In accordance with a first aspect of the present application, sub-picture extraction is rendered less complex by providing each slice with start position information which indicates a starting position from which onwards the picture is, along a coding path, encoded into the respective slice, relative to an onset position at which the coding path starts traversing a segment the starting position is located in. The picture is subdivided into the segments which are traversed by the coding path sequentially, segment by segment, with each slice having a fraction of a segment, or one or more segments completely, encoded therein, wherein the picture is encoded into the segments without coding interdependencies. The picture is coded into the datastream along the coding path in units of slices into which the picture is partitioned. Additionally, each slice comprises a segment information which indicates the segment the starting position is located in. Thus, start position information and segment information together enable to determine within which segment the starting position of a respective slice is located in, and where in this segment. Owing to the segment being encoded without coding interdependencies, it is possible to remove one or more slices concerning one segment without affecting the decodability of another segment. And even if the segments of the picture are used to compile a new sub-picture by shuffling or rearranging the segments and/or discarding some of the segments of the picture, it is possible

to form a respective sub-picture-specific datastream on the basis of the original datastream by discarding slices having none of the segments encoded therein which form the sub-picture, and adopting slices not having been discarded which still refer to the correct segment and, owing to the relative position indication, indicate the correct position of the starting position of the respective slice within the respective segment. Thus, in accordance with a first aspect of the present application, the datastream enables easy sub-picture extraction without the need for revising the start position information and segment information despite the rearrangement of segments and/or omission of some segments of the picture in the sub-picture datastream. In other words, the sub-picture extraction process is rendered easier by having the opportunity to merely leave-out or discard slices not pertaining to any segment contained in the sub-picture to be extracted, with adopting the remaining slices without having to revise the start position information.

[0143] In accordance with an embodiment, the start position information comprises a variable-length-coded start address syntax element. The variable-length coding may be used without any penalties during sub-picture extraction since the start position information needs not to be reversed anyway during sub-picture extraction.

[0144] In accordance with even further embodiments, the datastream is, by way of a start position coding mode flag, switchable between the start position information indicating the starting position of a respective slice relative to the segment's onset position, or absolutely relative to a picture onset position at which the coding path starts traversing the picture. In accordance with this option, an existing codec using the absolute start position indication so far, may be extended so as to take advantage of the relative position indication discussed herein. A decoder might be able to understand both types of start position indications offered by way of the start position information, or merely one of them.

[0145] In accordance with an even further embodiment, the datastream further comprises a base address data field defining, for each segment, a base address of the onset position of the respective segment which addresses the onset position along the coding path relative to the picture onset position. The conveyance of these base address reuses the computational overhead for computing the base address on the basis of the subdivision of the picture into the segments on its own.

[0146] In accordance with an embodiment, the segment information comprises a segment syntax element having a segment index encoded therein, which indexes the segment the starting position is located in. The syntax element might be encoded using a fixed length code. The datastream may comprise an index data field defining an association of a segment index value to each segment. In other words, the index data field may explicitly associate tags, namely segment index values, to the segments and the segment information of a slice is set to the tag or segment index value associated with the segment the starting position of the respective slice is located in. This tagging may easily be modified during sub-picture extraction.

[0147] Another aspect of the present application which may be combined with the first aspect is to render easier sub-picture extraction processes by treating sub-picture extraction processes similar to choosing one layer out of a multi-layer datastream. That is, according to the second aspect, an extractable datastream comprises a parameter set

indicating, not only, a size of the picture of the datastream and an indication of a first decoder capability level that may be used for decoding the picture from the datastream, but also at least one sub-picture decoding option and, for this at least one sub-picture decoding option, a reduced picture size and a second decoder capability level that may be used for decoding the sub-picture from an extracted version of the datastream. This sub-picture decoding option is treated like a sub-layer relative to a higher layer corresponding to the complete picture: the extracted version of the datastream results from the datastream by stripping off or discarding slices from the datastream having segments encoded there into which are spatially offset to the segment cluster which the sub-picture is composed of. The slices may or may not comprise the start position information and segment information as discussed above with respect to the first aspect. It may suffice, however, if each slice comprises a segment information which indicates the segment the respective slice covers. Thus, in accordance with the second aspect, the process of sub-picture extraction is like transitioning between several layers of a multi-layer datastream and does merely involve an omission of slices not pertaining the intended layer, i.e. here slices not covering any segment included into the sub-picture to be extracted. No “translation” or “modifying” the slices of the datastream to be adopted into the extracted datastream is necessary.

BRIEF DESCRIPTION OF THE DRAWINGS

[0148] Embodiments of the present invention will be detailed subsequently referring to the appended drawings, in which:

[0149] FIG. 1 shows a schematic diagram illustrating the concept of an extractable datastream using relative start position indication for slices for alleviating the sub-picture extraction process, wherein FIG. 1 illustrates the not-yet extracted datastream as well as the extraction process and extracted datastream as well as the participating entities including encoder, decoder and, optionally, a network device;

[0150] FIG. 2 shows a schematic diagram illustrating a picture and a fraction of the datastream in order to illustrate an embodiment according to which the datastream is switchable between relative slice start position indication and absolute slice start position indication;

[0151] FIG. 3 shows a schematic diagram illustrating an implementation of a datastream as presented with respect to FIG. 1 as a variant of an HEVC datastream;

[0152] FIG. 4 shows a pseudocode for illustrating the content conveyed within the parameter set, here exemplarily the picture parameter set, as it could be used in the implementation of FIG. 3;

[0153] FIG. 5 shows a schematic diagram of a picture and its segmentation into segments along with parameters used in pseudocodes presented with respect to the implementation examples provided with respect to FIG. 3;

[0154] FIG. 6 shows a pseudocode indicating the possible content conveyed in slice headers in accordance with an implementation example of FIG. 3;

[0155] FIG. 7 shows a pseudocode indicating a tiling syntax which could be used for implementing the codec example of FIG. 3;

[0156] FIG. 8 shows a pseudocode illustrating a possible content of a parameter set, here exemplarily a sequence parameter set, using the tiling syntax of FIG. 7;

[0157] FIG. 9 shows a pseudocode for illustrating a possible content of a parameter set, here exemplarily a picture parameter set using the tiling syntax of FIG. 7;

[0158] FIG. 10 shows a pseudocode illustrating a possible content of a content of a slice header for implementing the example of FIG. 3, differing from FIG. 6 in that segments may be compositions of one or more tiles;

[0159] FIG. 11 shows a pseudocode illustrating a possible content of a picture parameter set, here exemplarily a sequence parameter set, for illustrating the possibility that the parameter set conveys information for more than the complete picture decoding option, but additionally at least one sub-picture decoding option.

DETAILED DESCRIPTION OF THE INVENTION

[0160] A first embodiment of the present application is described with respect to FIG. 1. FIG. 1 shows a datastream 10 and a sub-picture-specific datastream 12 derived therefrom by sub-picture extraction 14. Datastream 10 might have been generated by an encoder 16. In particular, encoder 16 has encoded into the datastream 10 a picture 18. FIG. 1 illustrates the picture 18 as one picture of a video 20, but it should be noted that embodiments of the present application are not restricted to video datastreams. Rather, the embodiments described herein may easily be transferred onto picture codecs. The following description, however, illustrates embodiments related to video coding.

[0161] In encoding picture 18 into datastream 10, encoder 16 obeys or uses a coding path 22 which traverses picture 18 from a picture onset position 24 which, as depicted in FIG. 1, may be located at the top left-hand corner of picture 18, towards, for instance, an opposite picture corner such as the bottom right-hand corner in order to continue with traversing another picture of video 20, for instance. The obedience or usage of the coding path 22 determines, for instance, the availability of spatially neighboring portions of picture 18, such as samples or parameters derived therefrom by encoder 18 such as syntax elements, for coding a current portion of picture 18. For instance, encoder 16 may use predictive coding. To this end, encoder 16 may predict a current portion such as its sample content or syntax elements describing the same, on the basis of spatially neighboring portions of picture 18 provided that these neighboring portions precede the current portion along coding path 22. Additionally or alternatively, encoder 16 may use other spatial coding dependencies for a current portion such as deriving an entropy context for coding a syntax element describing a current portion of picture 18 on the basis of neighboring portions of picture 18. In addition, encoder 16 uses coding path 22 in order to partition or packetize the data into which picture 18 is coded. To this end, encoder 16 subdivides the coded data into slices along coding path 22. Accordingly, each slice 26 in datastream 10 which contains data of picture 18 covers, or has encoded therein, a corresponding portion, i.e. a slice 28 of picture 18 wherein the slices 28 of picture 18 are traversed by coding path 22 sequentially without interleaving slices 28. In other words, coding path 22 traverses each of the slices 28 into which picture 18 is subdivided merely once.

[0162] As also depicted in FIG. 1, encoder 16 obeys, in coding picture 18 into datastream 10, also a subdivision of picture 18 into segments 30. FIG. 1 exemplarily illustrates a subdivision of picture 18 into four segments 30 arranged,

exemplarily, in a 2x2 array. The number of segments, however, is not critical and may be different. The subdivision of picture 18 in segments 30, however, may be such that segments 30 are arranged in rows and columns. The subdivision of picture 18 into segments 30 may be such that segments 30 completely cover gaplessly picture 18 with segments within one segment row being of the same height, and segments within one segment column being of the same width. However, the thus rectangular segments 30 may vary in size in that the segment columns and/or segment rows may differ in width and height, respectively. The obedience of the subdivision of picture 18 into segments 30 may relate to the coding interdependencies: encoder 16 encodes picture 18 into datastream 10 in such a manner that coding interdependencies do not cross segment boundaries. To this end, no portion within a segment 30 of picture 18 is coded dependent on a portion of picture 18 external to this segment. In other words, encoder 16 encodes segments 30 independent from each other. As far as the relationship between slices 28 on the one hand and segments 30 on the other hand is concerned, slices 28 either completely cover one or more segments 30 or are merely within one segment 30 only. Merely for illustration purposes, FIG. 1 shows that each segment 30 is composed of two slices 28. This, in turn, means that coding path 22 traverses segments 30 sequentially, i.e. each segment 30 is traversed by coding path 22 completely before coding path 30 traverses a next segment 30 in coding order.

[0163] Thus, encoder 16 encodes picture 18 in units of slices 28 into datastream 10. Each slice 26 in datastream 10 has, thus, the corresponding picture slice 28 encoded thereinto. Each slice 26 comprises payload data 32 which encodes the content of the corresponding slice portion 28 of picture 18 in form of, for instance, one or more of prediction parameters such as prediction mode distinguishing, for instance, between intra or inter-coding mode, motion parameters for intra-predicted blocks, intra-prediction sub-modes for intra-predicted blocks, subdivision information for subdividing picture 18 into blocks, and residual data such as transform coefficients which represent the prediction residual.

[0164] Before proceeding with the description of the slices' 26 slice headers 34 and its components or syntax elements, respectively, the description of how encoder 16 codes picture 18 into datastream 10 shall briefly be continued. As mentioned previously, segments 30 are segments which are coded independently from each other. So far, however, the description focused on the encoding of picture 18 only. In case of picture 18 being a picture of a video 20, however, it might be that encoder 16 adopts the picture's subdivision into segments 30 for a sequence of pictures of video 20, i.e. subdivides the pictures of the picture sequence in the same manner, i.e. so that the other pictures are segmented into the same number of segments 30 and equally sized segments 30 with the segment borders coinciding spatially among the pictures. This is the situation depicted in FIG. 1, for instance, where a preceding and succeeding picture relative to picture 18 are illustrated as being subdivided in the same manner as picture 18 is, namely into four segments 30. In addition to the mutual independency of coding segments 30 within one picture, encoder 16 may encode each segment of picture 18 in a manner so that the encoding of such a respective segment 30 of picture 18 does not depend on portions of another picture, called reference

picture, which are outside or spatially offset to the segment collocated to the respective segment of picture 18. In other words, collocated segments 30 of the pictures within a picture sequence of video 20 may form a spatiotemporal region within video 20 within which encoder 16 performs the encoding independent from the spatiotemporal region formed by another segment of these pictures. Accordingly, the afore-mentioned inter-prediction, i.e. the motion-compensated prediction, would be restricted in a manner so that a block within one of segments 30 of picture 18 which is coded using inter-prediction would not refer, by way of a motion vector, to portions of a reference picture lying outside that segment of the reference picture which is collocated to the segment which the current block is located in. Rather, encoder 16 would, for instance, choose intra-prediction for that block instead.

[0165] And even further, it should be noted that, in addition to the slices' 26 slice headers 34 which are described in more detail below, datastream 10 comprises further information containing higher level coding parameters which is called parameter set in the following. This parameter set 36 may be distributed over datastream 10 and may be defined hierarchically in terms of scope. In other words, different portions of parameter set 36 may have a different scope. To this end, some coding parameters may be valid for one picture only such as picture 18, and others may represent coding parameters valid for a sequence of pictures including, for instance, picture 18, and even others may hold true for the whole video 20. Coding parameters defined in parameter set 36 will also be discussed in the following so that a brief discussion may suffice for the time being: coding parameters contained in parameter set 36 may, for instance, indicate the picture size of pictures of video 20, the picture's subdivision into segments 30, an indication of a decoder capability level that may be used for decoding picture 18 and video 20, respectively, from datastream 10 and/or decoding time stamps, coded picture buffer retrieval times and/or buffer sizes to be used for decoding datastream 10.

[0166] In addition to the details described so far, it should be noted that encoder 16 may form datastream 10 in a manner so that the slices 26 contained in datastream 10 which pertain one picture such as picture 18 are not interleaved with other slices 26 in datastream 10 pertaining another picture of video 20. Rather, they form a contiguous portion of datastream 10, a so-called access unit 36. An access unit 36, thus, summarizes the slices 26 concerning one picture such as picture 18. Within one access unit 36 other data, in addition to slices 26, may be contained as well such as, for instance, a portion of parameter set 36. The data within one access unit 36 may adhere to some predefined order or prescription for the ordering so that, on the basis of a surveillance of the order of data within datastream 10, the recipient of datastream 10 such as a decoder 38 may recognize the start of a next access unit in datastream 10. Even the slices 26 within one access unit 36 may be ordered mutually according to the order at which coding path 22 traverses the corresponding slices 28 within picture 18.

[0167] After having described the general framework for the coding of datastream 10, the description proceeds with a description of the slice header 34 and its content. In particular, encoder 16 provides each slice header 34 with a start position information which indicates a starting position 40 from which onwards the picture 18 is, along the coding path 22, encoded into the respective slice 26 comprising the

respective slice header 34. The knowledge about the starting position 40 is to enable decoder 38 to correctly position the picture content encoded into the respective slice 26, namely the picture content pertaining to corresponding slice portion 28 of picture 18 even if any preceding slice concerning picture 18, concerning a slice portion 28 preceding along coding path 22, is missing in datastream 10.

[0168] Favorably, however, the start position information 42 indicates the starting position 40 of the slice 26 within the slice header 34 of which the start position information 42 is contained, in a manner relative to an onset position 44 at which the coding path 22 starts traversing the segment 30 the starting position 40 is located in as indicated by an arrow 46 in FIG. 1, exemplarily. The relative indication of the starting position 40 alleviates the sub-picture extraction 14 as explained further below.

[0169] In order to compensate for the relative indication of the starting position 40 by the start position information 42, and its associated ambiguity, the slice header 34 additionally comprises segment information 48 which indicates the segment 30 the starting position 40 of the slice 26 is located in to which the segment information 48 belongs. In the example of FIG. 1, for instance, the slice 28, which the starting position 40 is pointed to by arrow 46, is located within the segment 30 indexed by the capital letter “B” in FIG. 1 and accordingly, segment information 48 of that slice would indicate that this slice 26 belongs to segment B. Together, start position information 42 and segment information 48, thus, suffice to determine the absolute position of the starting position 40 within picture 18, i.e. the relative position or starting position 40 with respect to the origin, namely the picture onset position 24.

[0170] The decoder 38 which receives datastream 10 is able to decode the start position information 42 of a respective slice 26 from datastream 10 and to locate, using the start position information 42, the starting position 40 of the respective slice relative to the onset position 44 of the segment the starting position 40 is located in. Further, decoder 38 decodes from datastream 10 the corresponding segment information 48 of the respective slice 26 so that the decoder 38 knows about the segment 30 the onset position 44 belongs to relative to which the start position information 42 indicates the starting position 40. The decoder 38 is, thus, able to locate the onset position 44 on the basis of the parameter set 36 which indicates the subdivision of picture 18 into segments 30 and accordingly, combine the relative position indication of the starting position 40, indicated relative to the onset position 44, with the position of the onset position 44 within picture 18 which combination yields the absolute position of starting position 40 within picture 18 relative to the picture onset position 24. In order to alleviate the computational overhead for decoder 38, more detailed embodiments described further below indicate the possibility that the parameter set 36 or some other portion in datastream 10 such as supplemental enhancement information, may indicate the onset position 44 for each segment 30 expressively so that the decoder 38 may use this indication rather than computing the onset position of the segment 30 itself.

[0171] It has already been outlined above that the relative indication of the starting position 40, relative to onset position 44 of the corresponding segment 30, is a favorable circumstance. This will now be discussed in more detail. In particular, the favorable circumstance becomes clear when

inspecting a sub-picture extraction process 14. The latter process may be performed by any entity processing datastream 10. For instance, the extraction process may be performed by a network device 50 connected into the transmission path between encoder 16 and a decoder 52 receiving the extracted datastream 12. The decoder 52 might be identical to decoder 38, i.e. could be one which would have been able to decode the complete or original datastream 10, too, or could be a decoder which is able to decode the extracted datastream 12, but is unable to decode the complete datastream 10 owing to, for instance, a decoder capability level which is lower than the one of decoder 38. The extraction process 50 could, however, also be performed within the decoder 52/38 itself.

[0172] The extraction process 14 aims at forming a datastream 12 which has pictures 54 encoded therein, which, compared to pictures 18, are composed merely of a proper subset of the segments 30 of picture 18 and/or differ in the mutual positioning of segments 30 within the picture area of picture 18 and 54, respectively. FIG. 1 illustrates the case where the pictures 54 of datastream 12 are merely composed of one of the four segments 30 of the picture 18 of the original datastream 10, namely segment B, but this is merely an example and as long as a rectangular picture area of picture 54 results, picture 54 could alternatively be composed of more than one of the segments 30 of picture 18.

[0173] Remarkably, with respect to the slices 26 contained in datastream 10, the extraction process 14 merely involves a discarding or omission of slices 26 pertaining segments 30 of picture 18 not contained in picture 54, whereas slices 26 pertaining any segment 30 contributing to the composition of picture 44, is adopted or left-in in the extracted datastream 12 without any modification, especially without any modification of the start position information 42 and the segment information 48. In particular, as the start position information 42 indicates the starting position 40 of slices 26 the starting position falls into a segment 30 contributing to picture 54, is indicated relative to the onset position of the segment, this relative indication is still correct within the reduced datastream 12. Moreover, the segment information 48 indicates for all slices 26 adopted in extracted datastream 12 as to which segment the corresponding slice 26 belongs to, i.e. within which segment its starting position lies. This segment information 48 has also been used in the extraction process 14 in order to identify those slices 26 to be omitted or discarded.

[0174] The only information present in the extracted datastream 12 not yet having been discussed with respect to the original datastream 10 are those coding parameters associated with the modified composition of pictures 54 of the extracted datastream 12 on the basis of the selected set of segments 30. Several possibilities exist with respect to this topic. According to a first alternative, the extraction process 14 may involve a modification of the aforementioned parameter set 36 so that the corresponding parameter set 36' in the extracted datastream 12 is modified to reflect the modified picture size of pictures 54 of the extracted video 56, the subdivision of picture 54 into segments, the decoder capability level that may be used for decoding the extracted datastream 12 which might be lower than the decoder capability level that may be used for decoding a complete video 20 or pictures 18 from datastream 10, and/or the modified decoding time stamps, coded picture buffer retrieval times and/or buffer sizes for decoding the extracted

datastream 12. In accordance with an alternative embodiment, however, the aspect of using the relative start position indication within start position information 42 is combined with an aspect of the present application according to which even the parameter set 36' of the extracted datastream 12 may be left unchanged, namely by providing the parameter set 36 of the original datastream 10 with both information items: in accordance with this alternative, parameter set 36 would indicate all the coding parameters for decoding the whole picture 18 from datastream 10, and concurrently indicate all the coding parameters in order to decode picture 54 from datastream 10 or 12. The coding parameters indicated by parameter set 36 with respect to the extracted datastream 12 and its picture 54 could be, at least partially, indicated in parameter set 36 in a manner relative to, or differentially to, the coding parameters indicated by parameter set 36 for the decoding of picture 18 from datastream 10. For instance, as outlined in more detail below, parameter set 36 could indicate the subdivision of picture 18 into segments 30 so that the sizes of segments 30 are clear from this part of parameter set 36. The coding parameters of parameter set 36 related to picture 54 could rely on this knowledge and could simply indicate as to which segments contribute to picture 54, and as to which mutual arrangement of these contributing segments within picture 54 applies. Even the afore-mentioned base address indication for the onset positions 44 within picture 18 could be repeated in datastream 36 with respect to the picture 54 of the extracted datastream 12, too. And as further explained below, parameter set 36 could, in this manner, not only indicate one extraction option or sub-picture decoding option for datastream 10, but maybe several ones differing in composition of a picture 54 on the basis of segments 30 of picture 18. Thus, decoder 52 would, in case of the alternative where the parameter set 36' remains unchanged with respect to parameter set 36 in the original datastream 10, receive, for instance, a decoding option indication 58 indicating whether a picture decoding, i.e. a decoding of picture 18, is to be performed on datastream 12, or a sub-picture decoding, i.e. a decoding of picture 54, is to be performed on the datastream 12. In the case, where the non-used slices 26 have already been stripped off as part of a previously performed extraction process 14, this decoding option indication may not indicate the picture decoding as the slices are missing. Remember, however, that the extraction process 14 might be performed by decoder 52 itself which, in turn, might be equal to decoder 38. To this end, decoder 52 might receive the complete original datastream 10 and then, decide on the basis of the decoding option indication 58, to perform the extraction process 14 in terms of stripping-off non-used slices 26 itself, i.e. discarding same. The decoding option indication could, for instance, be an external signalization provided by an application or a user. Alternatively, the decoding option indication could be an inherent indication of the decoder's 52 decoder capability level which, in turn, excludes some of the decoding options of datastream 10 from being decodable by decoder 52 anyway. If the picture decoding with respect to the complete picture 18 is to be performed, the decoding takes place as discussed with respect to decoder 38. If the sub-picture decoding option is to be applied, i.e. decoder 52 is to decode picture 54 from datastream 12, then the decoding process is substantially the same with the following differences: decoder 52 would use the coding parameters of parameter set 36' specific for this sub-picture decoding, i.e. for picture

54. Further, decoder 52 would compute different base addresses for the onset positions 44 of the segments 30 within picture 54 if the computation is to be performed by the decoder 52 itself. The different computational result is a consequence of the difference arrangement of the segments 30 within picture 54. The start position information 42 which indicates the relative position of the slices' starting position 40 is the same as it is indicated relative to the onset positions 44 of the segments by start position information 42. If the onset positions 44 of the segments are indicated in parameter set 36' for the arrangement of segments within picture 54, however, decoder 52 may use these base addresses which are specific for the sub-picture decoding to obtain picture 54, instead of an own computation.

[0175] More details on how decoder 38/52 might operate in decoding an inbound data stream—might it be 10 or 12—are set out at the end of description with respect to FIG. 11.

[0176] In the following description, a concrete example for implementing the above-outlined embodiment is described. In doing so, it is assumed that the datastream comprises within parameter set 36 a start position coding mode flag 60 which switches between the relative start position indication 46 of the slices' starting positions 40 relative to segment onset positions 44 as described with respect to FIG. 1, on the one hand, and an absolute start position indication 62 of the slices' starting positions 40 relative to the picture onset position 24. In other words, flag 60 signals whether the start position indications 42 in the slices 26 of datastream 10 indicate the starting positions 40 in the manner outlined with respect to FIG. 1, i.e. in relative terms with respect to the segments' onset positions 44, or absolutely with respect to the picture onset position 24. Flag 16 may, as further discussed below, then also be used in order to switch-on the presence of segment indication 48. Merely in case of the relative start position indication 46, the segment indication 48 might be present in the slices 26.

[0177] A further detail which will become clear from the subsequently explained embodiments is the fact that the slice position indication 42 might in fact be signaled by way of a combination of a flag and a conditionally signaled slice address: in case of the relative indication option 46, a flag contained by the start position information 42 would indicate whether the corresponding slice is the first in a segment, i.e. its starting position 40 coincides with the onset position 44 of the corresponding segment. If this is the case, the start position information 42 does not contain any start address syntax element. If not, however, the address syntax element points from the onset position 44 of the corresponding segment, to the starting position 40 of the slice. In case of the absolute indication option 62, the flag of the start position information 42 indicates whether or not the corresponding slice starts at the picture onset position 24, i.e. is the first slice of the picture, and if yes, the slice position information does not contain any address syntax element. If not, however, the address syntax element indicates the starting position 40 of the respective slice relative to the picture onset position 24. The just-mentioned flag comprised by start position information 42 might be used by decoders 38/52 for access unit border detection, i.e. detection of a new access unit 36, in case of absolute indication mode. To compensate for the inability to use this detectability in the relative indication mode, encoder 16 may use alternative access unit borer mechanisms such as the placement of access unit

delimiters, i.e. certain packets, at borders between consecutive access units. The decoder may then use these delimiters in case of relative indication mode.

[0178] And as a further note before starting with the subsequent more detailed explanation of embodiments, it is noted that different possibilities exist for indicating the starting position 40 by way of the start position information 42. For instance, picture 18 might be subdivided regularly into an array of blocks as illustrated at the top left-hand corner of picture 18 in FIG. 1. Coding path 22 may traverse these coding blocks 64 sequentially without interleaving, i.e. one coding block 64 is traversed completely before proceeding to the next coding block 64 of picture 18. The starting position information 42 may indicate position 42 measured in units of traversed coding blocks 64 up to the starting position 40 along coding path 22. If indicated relatively 46, start position information 42 would indicate the number of coding blocks 64 separating, along coding path 22, onset position 44 and slice starting position 40. In case of the absolute indication option 62, start position information 42 indicates the starting position 40 in units of traversed coding blocks 64 along coding path 22, when starting from the picture onset position 24. Accordingly, in this case the start position indication 42 would use a linear, one dimensional addressing domain and the base address of the onset position 44 of the segments mentioned above, could also be measured in number of coding blocks 64 traversed when starting from picture onset position 24 to the respective segment onset position 44. The summation of a base address 44 and the slice starting position 40 would yield the absolute address measured from the starting position 24 onwards. Alternative ways of signaling the starting position 40 by way of the start position information 42 would be feasible, however, too. For instance, the relative position of a starting position 40 relative to the corresponding segment onset position 44 could be indicated as x and y coordinates of a vector pointing from onset position 44 to the slice start position 40, with the components x and y measured in samples or units of coding blocks 64, respectively. Further, the addressing could be performed following a strictly row-wise linear addressing of the afore-mentioned coding blocks 64, i.e. in a manner where the addresses of the coding blocks within one complete row of picture 18 follow each other before the addresses increase with a first coding block of the next row within a picture, irrespective of the segmentation into segments 30. Internally, decoders 38 and 52 may switch between different address systems or domains as needed.

[0179] Another possibility which will become clear from the following description is the following: segments 30 have been described so far as being possible spatial cross-sections of spatiotemporal regions where the coding interdependencies do not cross the borders of these spatiotemporal regions. These segments 30 may, for instance, be defined to be sets of one or more tiles into which pictures 18 may be subdivided and which are coded independent from each other as far as the coding of one picture 18 is concerned. Thus, segments 30 may be tiles in case of one segment 30 being composed of one tile and in the subsequently explained variant A this is the case, whereas variant B assumes that segments 30 may be composed of one or more tiles. Moreover, according to the subsequent explanation, the afore-mentioned coding blocks 64 are CTUs (coding tree units) meaning that these blocks 64 are tree-root blocks further

subdivided hierarchically by multi-tree subdividing into coding blocks, prediction blocks and/or transform blocks in units of which encoder 16 selects the coding modes to be used, namely inter or intra-coding of corresponding coding blocks, in units of which the prediction parameters of the corresponding selected coding mode is set, namely for each prediction block which might be a leave node of a coding block, and in units of which the transform of the prediction residual takes place, namely in units of transform blocks which might be also leave blocks of coding blocks. The subdivision of CTUs 64 into coding units, prediction blocks and transform blocks might be signaled as part of the corresponding slice payload 32.

[0180] Thus, in the following, an implementation of the embodiment described with respect to FIG. 1 is illustrated as a possible modification of the HEVC framework. As described in the introductory portion of the specification of the present application, in HEVC DAM3 MCTS extraction, adjustment to the slice segment address of each slice header is useful, which in turn may impact even the byte alignment of each slice header owing to the variable length coding of the slice addresses. In particular, if using HEVC DAM3 MCTS, the slice addresses of slices adopted in an MCTS extracted datastream would have to be amended owing to their absolute indication option, i.e. they would have to be modified to refer to the new picture onset position of picture 54, and this could, owing to the variable length coding, lead to a different length of the slice address and, accordingly, to a different byte alignment. The subsequently explained modification of the HEVC framework would overcome this. In particular, a comparatively simpler MCTS sub-stream extraction results by signaling a part of the slice segment address to implicit or explicit association of slices with tiles/segments.

[0181] In the following, a first variant, variant A, of a modification of HEVC framework is described. As outlined above with respect to FIG. 1, according to this variant, the slice segment addresses are signaled relative to the first CTU of the respective tile or composition of tiles, i.e. the current segment 30, as opposed to signal slice segment addresses relative to the first CTU of the picture. "First" means, when using the HEVC coding order, the left-most and top-most CTU.

[0182] The difference between the two references, i.e. first CTU of the current segment 30, or first CTU of the picture, can be derived by the decoder, i.e. decoder 38 or 52, by combining the picture and tile instruction information from parameter sets 36 and 36', respectively, the association of slices 28/26 to this segment 30 which may be tiles or sets of tiles wherein the association may be transmitted in the datastream explicitly as outlined below.

[0183] Arrays with information about the segment sizes in CTBs and slice address offsets per segment 30 may be determined on decoder side. The access unit 36 start detection to be performed by a decoder 38 or 52, respectively, which may be performed on the basis of the flag contained in the start position information 42 in case of the absolute indication option 62 as this flag indicates the first slice of the access unit 36 and picture 18, respectively, may be done in case of the relative indication option 46 by use of an access unit delimiter NAL unit in the datastream 10, i.e. special packets which separate consecutive access units in datastream 10 are used to signal junctions between consecutive access units. As described above, the first_slice_in_pic_

flag may be comprised by the start position information 42 and may be turned into a flag that indicates the start of a segment 30 which, in turn, may be a tile or a composition of tiles.

[0184] FIG. 3 provides an over view of the resulting embodiment using the HEVC nomenclature. As indicated, the parameter set 36 is distributed over picture parameter sets 70, sequence parameter sets 72 and video parameter sets 74 differing in scope. FIG. 3 merely shows a fraction out of datastream 10, namely two VCL NAL units 76 and 78, each comprising an NALU header 80 indicating that same comprises slice information, followed by the corresponding slice 26 composed of slice header 34 and slice payload 32. The slice headers 34 refer to the corresponding picture parameter set 70 as indicated by arrows 80 and the picture parameter sets 70, in turn, point to the corresponding sequence parameter set as indicated by 82, and the sequence parameter sets 72, in turn, point to the corresponding valid video parameter set as indicated by 84. The parameter set 36 suggests, for instance, the picture parameter set 70 or the sequence parameter set 72, contains tiling syntax 86, i.e. syntax defining the subdivision of picture 14 into tiles 88 which, according to one alternative, form the segments 30 discussed above in FIG. 1. Thus, on the basis of the tiling syntax 86, it is possible for the decoder, for instance, to compute the base slice segment addresses as indicated by arrow 90. A summation 92 of the base slice segment addresses 94 and the offset slice segment addresses 96 conveyed as part of the start position information 42 and the slice headers 34 yields the corresponding slice segment address 98 of the corresponding slice 26, i.e. the slice segment address measured absolutely against the picture onset position 24. FIG. 3 shows the two slices 26 depicted in FIG. 3 as belonging to one tile of the tiles 88, namely tile number 2 which in fact is derivable for the decoder on the basis of this segment information 48 also comprised by the slice headers 34 as will become clear from the following description.

[0185] In particular, according to the embodiment shown in FIG. 4, the tiling structure or tiling syntax 86 may be comprised in the picture parameter set 70. It is noted, that an alternative embodiment would be feasible as well according to which the tiling syntax 86 would be present in the sequence parameter set 72, for instance, rather than the picture parameter set

[0186] In particular, as depicted in FIG. 4, a portion 100 of parameter set 70 indicates the subdivision of picture 18 into

segments 30, here tiles. The number of segment columns is indicated at 102, the number of segment rows is indicated at 104 and a flag 106 optionally offers the opportunity to signal in the datastream that the segment columns' width and segment rows' height is set uniformly. If not present, or if flag 106 is not set, the widths of the segment columns are indicated at 108 individually and the heights of the segment rows are indicated individually at 110.

[0187] Further, an explicit tagging of the segments is provided if a flag slice_segment_base_addr_per_tile_enable_flag 112 is set, namely by way of signaling, explicitly, at 114, a segment index value tile_id_in_pps[i] per segment i 30 of picture 18. The segment indices are conveyed in the datastream, i.e. in parameter set 70, by using a default order among the regular array of segments 30 such as row-wise as illustrated by a dashed arrow in FIG. 1, 116.

[0188] The semantics of the syntax element in FIG. 4 are outlined in detail below:

[0189] slice_segment_base_addr_per_tile_enable_flag equal to 0 specifies that the variable CtbAddrInRs of a slice segment is derived solely from slice_segment_address. slice_segment_base_addr_per_tile_enable_flag equal to 1 specifies that derivation of CtbAddrInRs is based on slice_segment_address and a tile dependent base address. When slice_segment_base_addr_per_tile_enable_flag is not present, it is inferred to be equal to 0.

[0190] tile_id_in_pps[i] specifies the id of tiles in bitstream order. The value of tile_id_in_pps[i] shall be in the range of 0 to 255 and tile_id_in_pps[i] shall not have the same value as tile_id_in_pps[j] for i unequal to j. When tile_id_in_pps is not present, it is inferred to be equal to 0

[0191] It might be a constraint of bitstream conformance that when slice_segment_base_addr_per_tile_enable_flag is equal to 1, access unit delimiter NAL units are present in the bitstream and that tiles_fixed_structure_flag is equal to 1.

[0192] The following pseudo codes indicate as to how a decoder like decoder 38 or 52, may compute certain dimensions of segments 30 and the base addresses of segments 30, for instance, on the basis of the available information in the datastream.

[0193] The following describes a coding tree block raster and tile scanning conversion process.

[0194] The list colWidth[i] for i ranging from 0 to num_tile_columns_minus1, inclusive, specifying the width of the i-th tile column in units of coding tree blocks (CTBs), is derived as follows (pseudo code 1):

```

if( uniform_spacing_flag )
for( i = 0; i <= num_tile_columns_minus1; i++ )
colWidth[ i ] = ( ( i + 1 ) * PicWidthInCtbsY ) / ( num_tile_columns_minus1 + 1 ) - ( i *
PicWidthInCtbsY ) / ( num_tile_columns_minus1 + 1 )
else {
colWidth[ num_tile_columns_minus1 ] = PicWidthInCtbsY
for( i = 0; i < num_tile_columns_minus1; i++ ) {
colWidth[ i ] = column_width_minus1[ i ] + 1
colWidth[ num_tile_columns_minus1 ] -= colWidth[ i ]
}
}
colWidth[i]
num_tile_columns_minus1

```

is the width of the i^{th} segment column among column 0...
num_tile_columns_minus1 measured in coding tree blocks CTBs 64
is the number segment columns minus 1

[0195] The list `rowHeight[j]` for `j` ranging from 0 to `num_tile_rows_minus1`, inclusive, specifying the height of the `j`-th tile row in units of CTBs, is derived as follows (pseudo code 2):

```

if( uniform_spacing_flag )
    for( j = 0; j <= num_tile_rows_minus1; j++ )
        rowHeight[ j ] = ( ( j + 1 ) * PicHeightInCtbsY ) / ( num_tile_rows_minus1 + 1 ) -
            ( j * PicHeightInCtbsY ) / ( num_tile_rows_minus1 + 1 )
else {
    rowHeight[ num_tile_rows_minus1 ] = PicHeightInCtbsY
    for( j = 0; j < num_tile_rows_minus1; j++ ) {
        rowHeight[ j ] = row_height_minus1[ j ] + 1
        rowHeight[ num_tile_rows_minus1 ] -= rowHeight[ j ]
    }
}
rowHeight[i]          is the height of the  $i^{th}$  segment row among rows 0... num_tile_rows_minus1
                      measured in coding tree blocks CTBs 64
num_tile_rows_minus1 is the number segment rows minus 1

```

[0196] The list `TileId[ctbAddrTs]` for `ctbAddrTs` ranging from 0 to `PicSizeInCtbsY-1`, inclusive, specifying the conversion from a CTB address in tile scan to a tile ID, is derived as follows (pseudo code 3):

```

for( j = 0, tileIdx = 0; j <= num_tile_rows_minus1; j++ )
    for( i = 0; i <= num_tile_columns_minus1; i++, tileIdx++ )
        tileIdxTileIdMap[ tile_id_in_pps[ tileIdx ] ] = tileIdx
        for( y = rowBd[ j ]; y < rowBd[ j + 1 ]; y++ )
            for( x = colBd[ i ]; x < colBd[ i + 1 ]; x++ )
                TileId[ CtbAddrRsToTs[ y * PicWidthInCtbsY + x ] ] = tileIdx

```

`tileIdxTileIdMap[ ]` is construed to map the `N` segment index values contained in vector `tile_id_in_pps[ ]` to the `tileIdx` of that segment
`tile_id_in_pps[ ]` is a vector containing entries 0...`N-1` with `N` being the number of segments 30, with the i^{th} entry indicating the segment index values associated with the i^{th} segment in raster scan segment order 116
`tileIdx` indexes the segments 30 in raster scan segment order 116
`TileId[ i ]` is construed to map a CTB address `i`, i.e. an address of a block 64, measured in coding order, or along coding path 22, to the `tileIdx` of the segment 39 this CTB is located in
`CtbAddrRsToTs[ i ]` is a function which maps a rank `i` of a block 64 which is the i^{th} block 64 of picture 18 in a picture raster scan order leading row-wise through the blocks 64 of picture 18, to its address measured in coding order 22
`colBd[ i ]` is a vector containing entries 0...`N` with `N` being the number segment columns minus 1, with the i^{th} entry, with $i < N$ indicating the position of the left-hand side of the i^{th} , from left to right, segment column measured in block 64, and `Nth` entry indicating the picture width in number of blocks

[0197] Please note that all the parameters used in the pseudo codes so far, although they have been described as being used for determining the base addresses in case of full picture decoding, i.e. with respect to picture 18, may be used for determining the base addresses for the segments' onset positions in sub-picture 54 as well. Here, the parameters `N`, `colWidth[i]`, `num_tile_columns_minus1`, `rowHeight[i]`, `num_tile_rows_minus1`, `tileIdxTileIdMap[ ]`, `tile_id_in_pps[ ]`, `CtbAddrRsToTs[i]`, `colBd[i]` are specific for this sub-picture 54 and its composition by segments 30 and its subdivision into corresponding blocks 64, respectively. Similarly, all references to the coding path 22 and raster scan segment order 116 are applied onto the respective sub-picture. `tileIdxTileIdMap[ ]` guarantees the maintenance of the association between the segment index values `tile_id_in_sh`, which assume any of `tile_id_in_pps[ ]`, one the one hand and the segments of the sub-picture 54 and their indexing in raster scan segment order 116, i.e. `tileIdx`, on the

other hand. As a minor note, it should be noted that the syntax element suffix “_pps” has been chosen here in order to reflect the example of carrying the segment index value to segment association in the PPS, but as already outlined above, this is merely an example and the signalization could also be realized elsewhere such as in SPS or VPS. In line with the latter statement, alternatively, the suffix “_ps” has been used and a corresponding replacement is naturally also applicable to the present embodiment. Similarly, the following examples will reveal that the 1:1 correspondence between tiles and segments may given up. This had also been outlined already with respect to FIG. 1. Accordingly, corresponding subsequent embodiments use name components “Area” or “area_” or “_area” instead which will become clear from the following description. Further, as also mentioned in the description of FIG. 1, an explicit assignment of tags or segment index values to the segments could be left of in alternative embodiments, with deriving the

mapping between segment index values in segment information 48 and the segments in the sub-picture 54 implicitly such as by using, for instance, the index domain tileIdx, instead, in which case the transmission of such association in the parameter set 36 for the picture 18 could be left off. However, the explicit assignment of segment index values to the segments 30 of picture 18 alleviates the mixing of tiles of two separate data streams.

[0198] The list `TileSizeInCtbsY[k]` for `k` ranging from 0 to $((\text{num_tile_rows_minus1}+1)*(\text{num_tile_columns_minus1}+1)-1)$, inclusive, specifying the size of the `k`-th tile within the picture in units of CTBs, is derived as follows (pseudo code 4):

```
for( j = 0, k = 0; j <= num_tile_rows_minus1; j++ )
  for( i = 0; i <= num_tile_columns_minus1; i++, k++ )
    TileSizeInCtbsY [ k ] = colWidth[ i ] * rowHeight[ j ]
TileSizeInCtbsY [ i ] is a vector containing entries 0...N-1 with N being the number of
                      segments 30, with the  $i^{\text{th}}$  entry indicating the number of blocks 64
                      contained in the  $i^{\text{th}}$  segment in raster scan segment order 116
```

[0199] The list `TileSliceSegBaseAddr[k]` for `k` ranging from 0 to $((\text{num_tile_rows_minus1}+1)*(\text{num_tile_columns_minus1}+1)-1)$, inclusive, specifying the slice segment address offset in tile scan of the first slice segment of the `k`-th tile within the picture in bitstream order in units of CTBs, is derived as follows (pseudo code 5):

```
TileSliceSegBaseAddr[ 0 ] = 0
for( k = 1; k <= ( ( num_tile_rows_minus1 + 1 ) * ( num_tile_columns_minus1 + 1 ) - 1 ); k++ )
  TileSliceSegBaseAddr[ k ] = TileSliceSegBaseAddr[ k - 1 ] +
  TileSizeInCtbsY [ k - 1 ]
TileSliceSegBaseAddr[ i ] is a vector containing entries 0...N-1 with N being the number
                          of segments 30, with the  $i^{\text{th}}$  entry indicating the address of the
                          onset position 44 of the  $i^{\text{th}}$  segment in raster scan segment
                          order 116, measured as the number of blocks 64 along coding
                          path from the picture onset position 24 to the segment onset
                          position 44.
```

[0200] Here, “tile scan” denotes the scan 22 as depicted in FIG. 1. The `TileSliceSegBaseAddr` denotes the afore-mentioned base addresses of the onset positions 44 of the segments which, according to the present example, are equal to tiles. The computations denoted above may be used by decoders 38 and 52, respectively, to compute the base addresses. If a sub-picture decoding applies, the number of segments and their arrangement is to be adapted.

[0201] FIG. 6 illustrates as to how the slice segment headers, i.e. the afore-mentioned slice headers 34, could look like. In particular, here, the slice header 34 comprises a flag 120, namely `first_slice_segment_in_area_flag` which, together with a syntax element `slice_segment_address` 122 forms the start position information 42. Merely if flag 120 is not set, `slice_segment_address` is present and indicates the offset of the slice start position 40 from the onset position 44 of the corresponding segment/tile 30. The latter is indicated by the segment information 48 which comprises merely one syntax element 124, namely `tile_id_in_sh` which assumes values out of the domain of values defined in parameter set 70 of FIG. 4, namely by way of the syntax elements 114.

[0202] In particular, the semantics could be as indicated below:

[0203] `first_slice_segment_in_area_flag` equal to 1 specifies that, when `slice_segment_base_addr_per_tile_enable_flag` is equal to 0, the slice segment is the first slice segment of the picture in decoding order. Otherwise, when `slice_segment_base_addr_per_tile_enable_flag` is equal to 1, `first_slice_segment_in_area_flag` equal to 1 specifies that the slice segment is the first slice segment of a tile of the picture in decoding order. `first_slice_segment_in_pic_flag` equal to 0 specifies that the slice segment is not the first slice segment of the picture in decoding order or tile in decoding order respectively. Thus, `slice_segment_base_addr_per_tile_enable_flag` acts as an example for flag 60 discussed with respect to FIG. 2.

[0204] NOTE 1—This syntax element may be used for detection of the boundary between coded pictures that are consecutive in decoding order. However, when IDR pictures are consecutive in decoding order and have the same NAL unit type, loss of the first slice of an IDR picture can cause a problem with detection of the boundary between the coded pictures. This can occur, e.g., in the transmission of all-intra-coded video in an error-prone environment. This problem can be mitigated by alternately using the two different IDR

NAL unit types (IDR_W_RADL and IDR_N_LP) for any two consecutive IDR pictures. The use of the temporal sub-layer zero index SEI message can also be helpful, as that SEI message includes the syntax element `irap_pic_id`, the value of which is different for IRAP pictures that are consecutive in decoding order. Some system environments have other provisions that can be helpful for picture boundary detection as well, such as the use of presentation timestamps in Rec. ITU-T H.222.0/ISO/IEC 13818-1 systems, access unit framing in the ISO/IEC 14496-12 ISO base media file format, or the marker bit in IETF RFC 3550 real-time transport protocol headers.

[0205] `dependent_slice_segment_flag` equal to 1 specifies that the value of each slice segment header syntax element that is not present is inferred to be equal to the value of the corresponding slice segment header syntax element in the slice header. When not present, the value of `dependent_slice_segment_flag` is inferred to be equal to 0.

[0206] The variable `SliceAddrRs` is derived as follows:

[0207] If `dependent_slice_segment_flag` is equal to 0, `SliceAddrRs` is set equal to `CtbAddrInRs`

[0208] Otherwise, `SliceAddrRs` is set equal to `SliceAddrRs` of the preceding slice segment containing the coding tree block for which the coding tree block address is `CtbAddrTsToRs[CtbAddrRsToTs[CtbAddrInRs]-1]`, where the variable `CtbAddrInRs` is specified in the semantics of `slice_segment_address`.

[0209] `tile_id_in_sh` specifies the index of the tile that the slice segment belongs to. The value of `area_id_in_sh` shall be in the range of 0 to 255. When `area_id_in_sh` is not present, it is inferred to be equal to 0. No more than one tile within a picture shall have the same value of `tile_id_in_sh`.

[0210] `slice_segment_address` specifies the address of the first coding tree block in the slice segment, in coding tree block raster scan of a picture (when `slice_segment_base_addr_per_tile_enable_flag` equal to 1) and tile scan of a picture otherwise as follows:

[0211] The variable `maxNumCtbY` is defined as $(\text{slice_segment_base_addr_per_tile_enable_flag} ? \text{TileSizeInCtbsY}[\text{tileIdxTileIdMap}[\text{tile_id_in_sh}]] : \text{PicSizeInCtbsY})$; here, `tileIdxTileIdMap[tile_id_in_sh]` maps the `tile_id_in_sh` of a currently decoded/encoded slice, comprised by its segment information 48, onto the correct `tileIdx`, i.e. references the correct segment in picture 18 or 54, and `TileSizeInCtbsY` yields for this segment the maximum of the relative slice address, namely `maxNumCtbsY-1`, to be represented/representable by `slice_segment_address` of its start position information 42. The decoder and encoder may use pseudo codes 1 to 4 to compute this information.

[0212] The length of the `slice_segment_address` syntax element is then $\text{Ceil}(\text{Log } 2(\text{maxNumCtbsY}))$ bits. The value of `slice_segment_address` shall be in the range of 0 to `maxNumCtbsY-1`, inclusive. When `slice_segment_base_addr_per_tile_enable_flag` is equal to 0, the value of `slice_segment_address` shall not be equal to the value of `slice_segment_address` of any other coded slice segment NAL unit of the same coded picture. When `slice_segment_base_addr_per_tile_enable_flag` is equal to 1, the value of `slice_segment_address` shall not be equal to the value of `slice_segment_address` of any other coded slice segment NAL unit belonging to the same tile within the same coded picture. When `slice_segment_address` is not present, it is inferred to be equal to 0.

[0213] The variable `CtbAddrInRs`, specifying a coding tree block address in coding tree block raster scan of a picture, is set equal to `CtbAddrTsToRs[slice_segment_address+(slice_segment_base_addr_per_tile_enable_flag?TileSliceSegBaseAddr[tileIdxTileIdMap[tile_id_in_sh]]: 0)]`. The variable `CtbAddrInTs`, specifying a coding tree block address in tile scan, is set equal to `CtbAddrRsToTs[CtbAddrInRs]`. The variable `CuQpDeltaVal`, specifying the difference between a luma quantization parameter for the coding unit containing `cu_qp_delta_abs` and its prediction, is set equal to 0. The variables `CuQpOffsetCb` and `CuQpOffsetCr`, specifying values to be used when determining the respective values of the `QpCb` and `QpCr` quantization parameters for the coding unit containing `cu_chroma_qp_offset_flag`, are both set equal to 0. Thus, in case of relative indication mode 46, `tileIdxTileIdMap[tile_id_in_sh]` maps the `tile_id_in_sh` of a currently decoded/encoded slice, comprised by its segment information 48, onto the correct `tileIdx`, i.e. references the correct segment in picture 18 or 54, and `TileSliceSegBaseAddr` yields for this segment the base address which is then to be added to the relative slice address, `slice_segment_address`, of its start position information 42 to yield to address of this slice. `CtbAddrTsToRs` merely translates the address measured along coding path 22 into the an address measured along a raster scan block order among blocks 64. The decoder and encoder may use pseudo codes 5 to compute the base addresses of the segment onset positions 44, or, in case of explicit information exemplified hereinafter, merely the encoder.

[0214] As to the order of VCL NAL units and their association to coded pictures the following may be said.

[0215] In particular, the following specifies the order of VCL NAL units and their association to coded pictures.

[0216] Each VCL NAL unit is part of a coded picture.

[0217] The order of the VCL NAL units within a coded picture is constrained as follows:

-
- The first VCL NAL unit of the coded picture shall have `first_slice_segment_in_pic_flag` equal to 1.
 - Let `sliceSegAddrA` and `sliceSegAddrB` be the values of `CtbAddrTsToRs[slice_segment_address ( slice_segment_base_addr_per_tile_enable_flag ? TileSliceSegBaseAddr[tileIdxTileIdMap[tile_id_in_sh]: 0 ] ) ]` of any two coded slice segment NAL units A and B within the same coded picture. When either of the following conditions is true, coded slice segment NAL unit A shall precede the coded slice segment NAL unit B:
 - `TileId[ CtbAddrRsToTs[ sliceSegAddrA ] ]` is less than `TileId[ CtbAddrRsToTs[ sliceSegAddrB ] ]`.
 - `TileId[ CtbAddrRsToTs[ sliceSegAddrA ] ]` is equal to `TileId[ CtbAddrRsToTs[ sliceSegAddrB ] ]` and `CtbAddrRsToTs[ sliceSegAddrA ]` is less than `CtbAddrRsToTs[ sliceSegAddrB ]`.
-

[0218] Decoders 38 and 52 may surveil these conditions in order to determine when a new access unit 36 starts in the inbound data stream 10 or 12, or to determine that any NAL unit is missing.

[0219] The variant A described so far may be varied in different terms to allow for an easier implementation and processing, for instance.

[0220] For instance, in the variant A, as already described above with respect to FIG. 1, segments 30 need not to be exactly tiles. To this, the slice segment base address calculation needs not to happen exclusively on tile granularity, but multiple tiles can be joined into an area then forming a segment 30 in accordance with the description brought forward above for which, then the slice segment base address may jointly be defined, i.e. slice addresses in slice headers are given relatively to the first CTU in bit stream order that belongs to a given area of multiple tiles, forming a segment 30.

[0221] Further, additionally or alternatively, in accordance with the variant B described in the following, it becomes clear that instead of burdening the client side or decoder with calculation of slice segment base addresses for each segment 30, an explicit signaling of slice segment base addresses may be provided in the datastream.

[0222] And even additionally or alternatively, the burden of repeating the tile structure signaling within each PPS of the video bit stream while guaranteeing, for instance, that tile_fixed_structure is equal to 1 when slice_segment_base_address_per_tile_enable_flag is equal to 1, may be reduced by allowing the tiling syntax to be carried either in SPS or PPS, wherein the PPS instance of the tiling syntax may override the tiling syntax setting carried within the SPS.

[0223] The following figures show by way of highlighting changes relative to the variant A described above. For instance, FIG. 7 shows a tiling syntax portion which may be used either in the PPS or SPS, wherein, when the tiling syntax of FIG. 7 is conveyed in a PPS, it overrules the tiling syntax provided by a corresponding SPS, i.e. it has higher precedents. FIG. 7, illustrates the possibility that the parameter set 36 of FIG. 1 carries bases addresses for the segment onset positions 44 by way of a base address data field 130.

[0224] The semantics could be as follows.

[0225] slice_segment_base_addr_per_area_enable_flag equal to 0 specifies that the variable CtbAddrInRs of a slice segment is derived solely from slice_segment_address. slice_segment_base_addr_per_area_enable_flag equal to 1 specifies that derivation of CtbAddrInRs is based on slice_segment_address and a tile dependent offset. When slice_segment_base_addr_per_area_enable_flag is not present, it is inferred to be equal to 0.

[0226] area_id_in_ps[i] specifies the id of tiles in bitstream order. The value of area_id_in_ps[i] shall be in the range of 0 to 255 and area_id_in_ps[i] shall not have the same value as area_id_in_ps[j] for i unequal to j. When area_id_in_sh is not present, it is inferred to be equal to 0

[0227] slice_segment_base_address[i] specifies that base slice segment address in tile scan order for all slice segments belonging to the area with the same value of area_id_in_ps[i]. When not present, the value of slice_segment_base_address[i] is inferred to be equal to 0.

[0228] It is a requirement of bitstream conformance that when area_id_enable_flag is equal to 1, access unit delimiter NAL units are present in the bitstream.

[0229] It is a requirement of bitstream conformance that for no two values of i, where area_id_in_ps[i] have equal value, the value of slice_segment_base_address[i-1] are not equal.

[0230] FIGS. 8 and 9 provide an example for SPS and PPS syntax with an individual enable flags for tiling syntax signaling. Here, the following semantics are used.

[0231] tiles_enabled_in_sps_flag equal to 1 specifies that the SPS carries the default tiling syntax describing the default structure of the CVS.

[0232] tiles_enabled_in_pps_flag equal to 1 specifies that the PPS carries tiling syntax describing the actual tiling structure of the pictures of the CVS that are referring to the PPS and overwriting the tiling syntax within the referred SPS.

[0233] The coding tree block raster and tiling scanning conversion process would then look as follows.

[0234] The list colWidth[i] for i ranging from 0 to num_tile_columns_minus1, inclusive, specifying the width of the i-th tile column in units of coding tree blocks (CTBs), is derived as follows (pseudo code 1'):

```

if( uniform_spacing_flag )
    for( i = 0; i <= num_tile_columns_minus1; i++ )
        colWidth[ i ] = ( ( i + 1 ) * PicWidthInCtbsY ) / ( num_tile_columns_minus1 + 1 ) - ( i * PicWidthInCtbsY ) / ( num_tile_columns_minus1 + 1 )
else {
    colWidth[ num_tile_columns_minus1 ] = PicWidthInCtbsY ( 6 3 )
    for( i = 0; i < num_tile_columns_minus1; i++ ) {
        colWidth[ i ] = column_width_minus1 [ i ] + 1
        colWidth[ num_tile_columns_minus1 ] -= colWidth[ i ]
    }
}

```

[0235] The list rowHeight[j] for j ranging from 0 to num_tile_rows_minus1, inclusive, specifying the height of the j-th tile row in units of CTBs, is derived as follows (pseudo code 2'):

```

if( uniform_spacing_flag )
    for( j = 0; j <= num_tile_rows_minus1; j++ )
        rowHeight[ j ] = ( ( j + 1 ) * PicHeightInCtbsY ) / ( num_tile_rows_minus1 + 1 ) - ( j * PicHeightInCtbsY ) / ( num_tile_rows_minus1 + 1 )

```

-continued

```

else {
    rowHeight[ num_tile_rows_minus1 ] = PicHeightInCtbsY (6 4)
    for( j = 0; j < num_tile_rows_minus1; j++ ) {
        rowHeight[ j ] = row_height_minus1[ j ] + 1
        rowHeight[ num_tile_rows_minus1 ] -= rowHeight[ j ]
    }
}

```

[0236] The list TileId[ctbAddrTs] for ctbAddrTs ranging from 0 to PicSizeInCtbsY-1, inclusive, specifying the conversion from a CTB address in tile scan to a tile ID, is derived as follows (pseudo code 3'):

```

for( j = 0, tileIdx = 0; j <= num_tile_rows_minus1; j++ )
    for( i = 0; i <= num_tile_columns_minus1; i++, tileIdx++ )
        for( y = rowBd[ j ]; y < rowBd[ j + 1 ]; y++ )
            for( x = colBd[ i ]; x < colBd[ i + 1 ]; x++ )
                TileId[ CtbAddrRsToTs[ y * PicWidthInCtbsY + x ] ] = tileIdx

```

[0237] The variables remain their meaning as taught above, with N denoting, however, the number of tiles which may, however, due to collection of tiles to segments **30**, be greater than the number M of segments **30**.

[0238] The list TileSizeInCtbsY[k] for k ranging from 0 to ((num_tile_rows_minus1+1)*(num_tile_columns_minus1+1)-1), inclusive, specifying the size of the k-th tile within the picture in units of CTBs, is derived as follows (pseudo code 4'):

```

for( areaId = 0; areaId <= 255; areaId ++ )
    AreaSizeInCtbsY [ areaId ] = 0
for( j = 0, k = 0; j <= num_tile_rows_minus1; j++ )
    for( i = 0; i <= num_tile_columns_minus1; i++, k++ )
        if ( area_id_in_ps[ j * ( num_tile_columns_minus1 + 1 ) + i ] == areaId )
            AreaSizeInCtbsY [ areaId ] += ( colWidth[ i ] * rowHeight[ j ] )

```

area_id_in_ps [] is a vector containing entries 0...N-1 with N being the number of tiles, with the i^{th} entry indicating the segment index value associated with the segment which the i^{th} tile in tile raster scan segment order is located in; the values entered in the entries assume values between 0...256 or, differently speaking, in the domain of possible values for the segment index in the segment information 48; remember the dependency on the picture decoding option/target, i.e. whether the base addresses of segments within picture 18 or sub-picture 54 are to be computed, so that N is the number of tiles in picture 18 or sub-picture 54, respectively

AreaSizeInCtbsY [] is construed to become a vector containing an entry for each member of the domain of possible values for the segment index in the segment information 48, with indicating for each segment index value occurring in picture 18 or sub-picture 54, whatever is the current decoding target/option, the number of blocks 64 covered thereby

[0239] A mapping from AreaId to AreaIdx is derived as follows:

```

NumTilesInArea[0...255]=0;
NumAreas = 1
for( k = 0; k <= ( ( num_tile_rows_minus1 + 1 ) * ( num_tile_columns_minus1 + 1 ) - 1 ); k++ )
    NumTilesInArea[ area_id_in_ps[ k ] ]+=1
    If( NumTilesInArea[ area_id_in_ps[ k ] ] == 1 )
        AreaIdToAreaIdx[area_id_in_ps[ k ]]= NumAreas++;

```

NumTilesInArea [] is construed to become a vector containing an entry for each member of the domain of possible values for the segment index in the segment information 48, with indicating for each segment index value occurring in picture 18 or sub-picture 54, whatever is the current decoding target/option, the number of tiles covered by the respective segment having this segment index value

AreaIdToAreaIdx [] is construed to become a vector containing an entry for each member of the domain of possible values for the segment index in the segment information 48, with indicating for each segment index value occurring in picture 18 or sub-picture 54, whatever is the current decoding target/option, its rank or index when assigned or measured along a raster scan segment order leading segment-row-wise from top left to bottom right

[0240] The list AreaSliceSegBaseAddr[k] for k ranging from 0 to (NumAreas-1), inclusive, specifying the slice segment address offset of the first slice segment of the k-th area within the picture in bitstream order in units of CTBs, is derived as follows (pseudo code 5'):

```
AreaSliceSegBaseAddr[ 0 ] = 0
for( k=1; k < ( NumAreas ); k++ )
    AreaSliceSegBaseAddr[ k ] =
        slice_segment_base_address[ k - 1 ]
AreaSliceSegBaseAddr[ i ] is construed to become a vector containing entries 0...M-1
with M being the number of segments 30, being equal to
NumAreas, and indicating at the ith entry the base address, i.e.
the segment onset position 44, for the ith segment in the afore-
mentioned raster scan segment order, i.e. using AreaIdx
which may be associated with area_id_in_sh of the slice's
segment information 48 by decoder and encoder using
AreaIdxToAreaIdx.
```

[0241] Note that, thus, the base address data field 130 which is defined by the syntax elements slice_segment_base_address, defines, for each segment 30, the base address

address offset of the first slice segment of the k-th area within the picture in bitstream order in units of CTBs, is derived as follows (pseudo code 5''):

```
TileSliceSegBaseAddr[ 0 ]=0
AreaSliceSegBaseAddr[ area_id_in_ps[ 0 ] ] = 0
NumTilesInArea[0...255]=0;
NumTilesInArea[ area_id_in_ps[ 0 ] ]=1
AreaIdxToAreaIdx[area_id_in_ps[ 0 ] ] = 0
NumAreas = 1
for( k = 1; k <= ( ( num_tile_rows_minus1 + 1 ) * ( num_tile_columns_minus1 + 1 ) -
1 ) ; k++ )
    NumTilesInArea[ area_id_in_ps[ k ] ]+=1
    TileSliceSegBaseAddr[ k ] = TileSliceSegBaseAddr[ k - 1 ] +
        TileSizeInCtbsY[ k - 1 ]
    If(NumTilesInArea[ area_id_in_ps[ k ] ] == 1)
        AreaSliceSegBaseAddr[NumAreas]=TileSliceSegBaseAddr[ k ]
        AreaIdxToAreaIdx[area_id_in_ps[ k ]]= NumAreas;
        NumAreas++;
```

of the onset position 44 of the respective segment which addresses the onset position 44 relative to the picture onset position 24, although the explicit transmission of a base address for the first segment 30 of the picture 18 or sub-picture 54, in case of applying the explicit transmission to sub-picture decoding option(s), too, in raster scan segment order is omitted, as same is zero anyway. The definition that this base address is zero by way of the base address data field is a result of the convention that the explicit base address transmission starts with the second segment in raster scan segment order. Naturally, an alternative embodiment could be assessed where explicit base address transmission is used for all segments instead.

[0242] In case of not using the explicit base address signalization, but using an implicit derivation of the base addresses instead, same may be performed by decoder or encoder as follows. The following computation is used by the encoder for the computation of the base addresses even in case of explicit base address signalization.

[0243] The list TileSliceSegBaseAddr[k] for k ranging from 0 to ((num_tile_rows_minus1+1)*(num_tile_columns_minus1)-1), inclusive, specifying the slice segment address offset of the first slice segment of the k-th tile within the picture in bitstream order in units of CTBs, and the list AreaSliceSegBaseAddr[k] for k ranging from 0 to (NumAreas-1), inclusive, specifying the slice segment

[0244] And additionally, FIG. 10 shows as to how the slice header could look like according to variant B with the following showing a possible semantics.

[0245] first_slice_segment_in_area_flag equal to 1 specifies that, when tiles_enabled_flag is equal to 0, the slice segment is the first slice segment of the picture in decoding order. Otherwise, when tiles_enabled_flag is equal to 1, first_slice_segment_in_pic_flag equal to 1 specifies that the slice segment is the first slice segment of a tile of the picture in decoding order. first_slice_segment_in_pic_flag equal to 0 specifies that the slice segment is not the first slice segment of the picture in decoding order.

[0246] NOTE 1—This syntax element may be used for detection of the boundary between coded pictures that are consecutive in decoding order. However, when IDR pictures are consecutive in decoding order and have the same NAL unit type, loss of the first slice of an IDR picture can cause a problem with detection of the boundary between the coded pictures. This can occur, e.g., in the transmission of all-intra-coded video in an error-prone environment. This problem can be mitigated by alternately using the two different IDR NAL unit types (IDR_W_RADL and IDR_N_LP) for any two consecutive IDR pictures. The use of the temporal sub-layer zero index SEI message can also be helpful, as that SEI message includes the syntax element irap_pic_id, the value of which is different for IRAP pictures that are consecutive in decoding order. Some system environments

have other provisions that can be helpful for picture boundary detection as well, such as the use of presentation timestamps in Rec. ITU-T H.222.0/ISO/IEC 13818-1 systems, access unit framing in the ISO/IEC 14496-12 ISO base media file format, or the marker bit in IETF RFC 3550 real-time transport protocol headers.

[0247] `dependent_slice_segment_flag` equal to 1 specifies that the value of each slice segment header syntax element that is not present is inferred to be equal to the value of the corresponding slice segment header syntax element in the slice header. When not present, the value of `dependent_slice_segment_flag` is inferred to be equal to 0.

[0248] The variable `SliceAddrRs` is derived as follows:

[0249] If `dependent_slice_segment_flag` is equal to 0, `SliceAddrRs` is set equal to `CtbAddrInRs`

[0250] Otherwise, `SliceAddrRs` is set equal to `SliceAddrRs` of the preceding slice segment containing the coding tree block for which the coding tree block address is `CtbAddrTsToRs[CtbAddrRsToTs[CtbAddrInRs]-1]`, where the variable `CtbAddrInRs` is specified in the semantics of `slice_segment_address`.

[0251] `area_id_in_sh` specifies the index of the tile that the slice segment belongs to. The value of `area_id_in_sh` shall

picture, is set equal to `CtbAddrTsToRs[slice_segment_address+(slice_segment_addr_offset_per_tile_enable_flag?TileSliceSegBaseAddr[AreaIdToAreaIdx[area_id_in_sh]:0]]`. The variable `CtbAddrInTs`, specifying a coding tree block address in tile scan, is set equal to `CtbAddrRsToTs[CtbAddrInRs]`. The variable `CuQpDeltaVal`, specifying the difference between a luma quantization parameter for the coding unit containing `cu_qp_delta_abs` and its prediction, is set equal to 0. The variables `CuQpOffsetCb` and `CuQpOffsetCr`, specifying values to be used when determining the respective values of the `Qp'Cb` and `Qp'Cr` quantization parameters for the coding unit containing `cu_chroma_qp_offset_flag`, are both set equal to 0.

[0256] As to the order of VCL NAL units and their association to coded pictures and access units 36, the following may be said:

[0257] Each VCL NAL unit is part of a coded picture.

[0258] The order of the VCL NAL units within a coded picture is constrained as follows:

- The first VCL NAL unit of the coded picture shall have `first_slice_segment_in_pic_flag` equal to 1.

- Let `sliceSegAddrA` and `sliceSegAddrB` be the values of `CtbAddrTsToRs[slice_segment_address + CtbAddrRsToTs[ (slice_segment_base_addr_per_tile_enable_flag ? AreaSliceSegBaseAddr[AreaIdToAreaIdx[area_id_in_sh]:0]:0 ) ] ]` of any two coded slice segment NAL units A and B within the same coded picture. When either of the following conditions is true, coded slice segment NAL unit A shall precede the coded slice segment NAL unit B:

- `TileId[ CtbAddrRsToTs[ sliceSegAddrA ] ]` is less than `TileId[ CtbAddrRsToTs[ sliceSegAddrB ] ]`.
- `TileId[ CtbAddrRsToTs[ sliceSegAddrA ] ]` is equal to `TileId[ CtbAddrRsToTs[ sliceSegAddrB ] ]` and `CtbAddrRsToTs[ sliceSegAddrA ]` is less than `CtbAddrRsToTs[ sliceSegAddrB ]`.

be in the range of 0 to 255. When `area_id_in_sh` is not present, it is inferred to be equal to 0.

[0252] `slice_segment_address` specifies the address of the first coding tree block in the slice segment, in coding tree block raster scan of a picture (when `slice_segment_base_addr_per_tile_enable_flag` equal to 1) and tile scan of a picture otherwise as follows:

[0253] The variable `maxNumCtbsY` is defined as $(\text{slice_segment_addr_offset_per_tile_enable_flag} ? \text{AreaSizeInCtbsY}[\text{area_id_in_sh}] : \text{PicSizeInCtbsY})$

[0254] The length of the `slice_segment_address` syntax element is $\text{Ceil}(\log_2(\text{maxNumCtbsY}))$ bits. The value of `slice_segment_address` shall be in the range of 0 to `maxNumCtbsY-1`, inclusive. When `slice_segment_addr_offset_per_tile_enable_flag` is equal to 0, the value of `slice_segment_address` shall not be equal to the value of `slice_segment_address` of any other coded slice segment NAL unit of the same coded picture. When `slice_segment_addr_offset_per_tile_enable_flag` is equal to 1, the value of `slice_segment_address` shall not be equal to the value of `slice_segment_address` of any other coded slice segment NAL unit of the same area within the same coded picture. When `slice_segment_address` is not present, it is inferred to be equal to 0.

[0255] The variable `CtbAddrInRs`, specifying a coding tree block address in coding tree block raster scan of a

[0259] The description of a possible HEVC codec extension towards efficient sub-picture extraction presented above is preliminarily extended with respect to the aspect already announced with respect to FIG. 1 according to which the sub-picture extraction process is not only alleviated due to advantageous relative indication of the slices' starting position with respect to the current segments' onset positions, but additionally by providing a parameter set in the datastream which does not only provide information with respect to the decoding of the whole picture/video, but also with respect to the decoding of the sub-picture decoding option or, alternatively, several sub-picture decoding options so that the extraction process may, in fact, merely be restricted to actions concerning the omission or discarding of slices not needed according to the sub-picture decoding option of interest. In even other words, according to this extension of the above-described HEVC modification, the interest in decoding merely a sub-picture, i.e. the selection of a sub-picture decoding option, is treated like the omission of layers of a multi-layer datastream with choosing all corresponding layers corresponding to a decoding of the complete picture area of the original datastream. Later on, it will be noted that the latter aspect may also be used independent from the relative slice starting point indication discussed above.

[0260] Thus, the extension described now allows for an easier extraction process without parameter set rewriting.

Like in extraction processes of layered bit streams, removal of NAL unit carrying slice data and parameter sets is sufficient to create a conforming bit stream and a decoder receiving the bit stream could easily determine the actual operation point represented in the extracted bit stream. The operation point in this context exists within a space spent, at least, by the subset of the segment grid or tiling grid in case of using tiles as segments, and, optionally, further scalability dimensions such as spatial resolution, temporal resolution, SNR scalability and so forth.

[0261] According to the current embodiment, parameter sets, such as VPS or SPS, define several operation points in tiling context, in the following referred to as output tile set (OTS), for which several characteristics are described. A specific embodiment is depicted in FIG. 11, here exemplarily assuming that the parameters concerning OTS are contained in the SPS.

[0262] As depicted in FIG. 11, the parameter set, here exemplarily the SPS, indicates how the decoding of the complete pictures 18 (compared FIG. 1) a tiling syntax 140 an example of which has been presented in FIG. 7 exemplarily and which indicates the subdivision of picture 18 into segments and, optionally, the association of segment index values to these segments and, optionally, the base addresses of the onset positions of these segments, respectively, as well as the size of picture 18, namely the horizontal size at 142 and the vertical size at 144 as well as the decoder capability level for decoding picture 18 at 146 for $i=0$, for instance. With `num_output_tile_format` being greater than 1, which syntax element is transmitted at 148, the parameter set of FIG. 11 additionally signals the information 142 to 146 for further sub-picture decoding options indicating other picture sizes and associated decoder capability levels for pictures 54. According to the example of FIG. 11, various compositions of segments 30, here exemplarily tiles, to result into corresponding picture sizes as indicated by 142 and 144 for the output formats is signaled, too, separately. `Num_output_tile_sets` is in syntax elements which is transmitted at 150 and indicates the number of search compositions. Each composition is associated with one of the output tile formats by an index 152. The number of tiles which compose the respective picture 18 or picture 54 is transmitted at 154 and the segment indices are transmitted for the individual tile compositions at 156. Finally, for each tile composition of the number indicated by 150, the parameter set indicates at 158 additional information for decoding, namely parameters related to decoding time stamps and/or coded picture buffer with travel times and/or buffer sizes needed for decoding the respective segment composition (or tile composition) from the datastream.

[0263] When using the embodiment of FIG. 11, the decoding process which takes place in decoder 38 or 52, takes an additional input parameter which may refer to as Target Output Tile Set, that indicates which of the defined OTS is selected for decoding, wherein this input parameter represents the signal 58 discussed above with respect to FIG. 1.

[0264] The parameter `TargetOutputTileSet` can determine by various means:

[0265] external means such as an indication by a system layer through signaling in File Format, Transport Stream, SDP or others.

[0266] the presence of an SEI message in the bitstream indicating which tiles or OTSs are present within the bitstream, i.e. a description of former extraction processes the bitstream underwent. Thus, extraction information may be contained in the data stream which indicates one or more decodable OTS.

[0267] parsing the values of syntax elements `tile_id_sh` in slice headers of NAL units within an access unit to determine which of the defined OTS can be decoder with contents of the bitstream.

[0268] A priority can be assigned to the defined OTSs to help determine the OTSs to be decoded, e.g. through:

[0269] explicit signalling in the loop over `num_output_tile_sets` or SEI messages assigning a priority to the OTSs

[0270] per order in the loop over `num_output_tile_sets`

[0271] present tiles/removed tiles SEI message with indication of all contained OTS in bitstream

[0272] Thus, the parameter set 36 may contain, for each decoding option as signalled by 148, and for each segment collection as defined by 150 and 158 for that decoding option, a priority indication indicating a priority at which this segment collection which defines a specific sub-picture composition, is to be chosen for the decoding target by decoder 38 and 52, respectively. The priority indication may indicate the priorities among all of the segment collection options the number of which is signalled by 150 globally, or merely individually among the groups belonging to one common sub-picture option, i.e. same sub-picture size and corresponding decoder capability level.

[0273] The `TileSliceSegBaseAddr` derivation needs to be adjusted (highlighted in turquoise) with respect to Variant A as follows:

[0274] The list `TileId[ctbAddrTs]` for `ctbAddrTs` ranging from 0 to `PicSizeInCtbsY-1`, inclusive, specifying the conversion from a CTB address in tile scan to a tile ID, is derived as follows:

```
for( j = 0, tileIdx = 0; j <= num_tile_rows_minus1; j++ )
  for( i = 0; i <= num_tile_columns_minus1; i++, tileIdx++ )
    tileIdxTileIdMap[ tile_id_in_pps[ tileIdx ] ] = tileIdx
    for( y = rowBd[ j ]; y < rowBd[ j + 1 ]; y++ ) ( 6 9 )
      for( x = colBd[ i ]; x < colBd[ i + 1 ]; x++ )
        TileId[ CtbAddrTsToTs[ y * PicWidthInCtbsY + x ] ] = tileIdx
```

[0275] The list `TileSizeInCtbsY[k]` for k ranging from 0 to $((\text{num_tile_rows_minus1}+1)*(\text{num_tile_columns_minus1}+1)-1)$, inclusive, specifying the size of the k -th tile within the picture in units of CTBs, is derived as follows:

```
for( j = 0, k = 0; j <= num_tile_rows_minus1; j++ )
  for( i = 0; i <= num_tile_columns_minus1; i++, k++ )
    TileSizeInCtbsY[ k ] = colWidth[ i ] * rowHeight[ j ]
```

[0276] For the i -th target OTS, the list `TileSliceSegBaseAddr[k]` for k ranging from 0 to $((\text{num_tile_rows_minus1}+1)*(\text{num_tile_columns_minus1}+1)-1)$, inclusive, specifying the slice segment address offset in tile scan of the first slice segment of the k -th tile within the picture in bitstream order in units of CTBs, is derived as follows:

```

TileSliceSegBaseAddr[ 0 ] = 0
cnt=1
for( k = 1; k <= ( ( num_tile_rows_minus1 + 1 ) * (num_tile_columns_minus1 + 1) -
1 ) ; k++ )
if ( tileIdxInTargetOTS ) {
    TileSliceSegBaseAddr[ cnt ] = TileSliceSegBaseAddr[ cnt - 1 ] +
    TileSizeInCtbsY [ k - 1 ]
    cnt++
    // with cnt derivable from tile_id_in_sh with a mapping table to
    tile_id_in_sps
}

```

[0277] Wherein `tileIdxInTargetOTS` is true for all tiles that belong to the `TargetOutputTileSet` and false otherwise. It is noted that with the above, the tile arrangement within the decoded picture of the `TargetOutputTileSet` remains similar to tile arrangement in the original bitstream. Raster to tile scan conversion and vice versa is done considering the tiles within the `targetOutputTileSet` only.

[0278] As already denoted above, the extension just described with respect to FIG. 11, which concerns the parameter set 36 and its signaling of picture size and corresponding decoder capability level not only for original picture 18, but also for sub-picture 54, may be used in a frame work where the afore-mentioned slices' start position information 42 is not present, for instance. For example, by convention, slices might be restricted to not start within a segment 30, but merely at segment onset positions. Then, the segment information 48 might suffice, for instance. Other examples might also be possible. In either case, the decoder 38 or 52 which receives stream 10 or 12 (with or without the start position information 42) uses the information in the parameter set 36 which equals 36' as follows. In particular, it decodes the parameter set 36 from the inbound data stream which might be 10 or 12 and derives the decoding option indication 58 whether a picture decoding or sub-picture decoding is to be performed on the inbound data stream. Picture decoding denotes the target of decoding picture 18 from data stream 10, i.e. the composition of picture 10 out of all segments 30. Hence, all slices 26 need to be in the data stream 10 to achieve this aim. A sub-picture decoding aim is the target of decoding a picture 54 from data stream 12. "A picture 54" means that more than one sub-picture decoding option may be signaled in parameter set 36 with same differing in picture size and shape and decoder capability level. For instance, in the example of FIG. 1, different sub-picture decoding options may result from forming composing a sub-picture 54 from two segments 30 side by side, composing a sub-picture 54 from two segments 30 one upon the other or composing a sub-picture 54 from one segment 30 only, with these examples being extendible by composing a sub-picture out of three segments 30, for example.

[0279] If the picture decoding is to be performed on the data stream, the decoder derives from the parameter set 36 a size of the picture 18 such as via 142 and 144 for $i=0$ and an indication of the decoder capability level that may be used for decoding the picture 18 such as via 146 for $i=0$ from the data stream, and it derives from the parameter set 36 an information on the subdivision of the picture 18 into a first set of segments 30, namely all segments, which are encoded into the data streams without coding interdependencies, such as from tiling syntax 140. Further, it decodes the picture 18 from the data stream in units of slices 26 into which the picture 18 is partitioned along coding path 22. As to the

placement of decoded picture content of the individual slices 26 in case of same comprising start position information, see the above description. It might be that the decoder checks the decoder capability level whether same is met by the decoder 38.

[0280] In case of the decoding option indication 58 suggesting that a sub-picture decoding is to be performed on the inbound data stream, however, the decoder derives from the parameter set 36 a further picture size of sub-picture 54 and an indication of a decoder capability level that may be used for decoding the sub-picture 54 from the data stream such as via 142, 144 and 146 for $i \neq 0$, derives from the parameter set 36 an information on the subdivision of the sub-picture 54 into a second set of segments which segments are encoded into the data streams without coding interdependencies and which second set is a subset of the set of segments of picture 18, and decodes the sub-picture 54 from the data stream in units of those slices 26 into which the sub-picture 54 encoded. Even here the decoder uses or obeys a coding path 22'. This time it traverses the other target aim, namely sub-picture 54. Like coding path 22 traversing picture 18, coding path 22' sequentially, segment by segment, traverses the sub-picture 54 and each slice 26 among those pertaining any fraction of sub-picture 54, has a fraction of a segment, or one or more segments completely, out of the second set of segments encoded thereinto, which form the composition of sub-picture 54. Note that coding path or order 22' might traverse the subset of segments 30 participating in composing sub-picture 54 in an order different from the order same are traversed by coding path 22 within original picture 18. This does, however, not influence the decoding result as the segment coding independency prohibits coding dependencies across segment boundaries anyway. Within each segment 30, however, paths 22 and 22' coincide, and that is important to keep decoding synchrony and even the placement of slices' content when using start position information 42 is not influenced. Again, decoder may check the decoder capability level for the sub-picture decoding option whether same is met by the decoder 38/52.

[0281] As discussed above, the decoder may derive the decoding option indication 58 by different means. For example, the decoder 52 may analyze the segments covered by the slices present in the inbound data stream, i.e. 10 or 12, to determine whether and if yes as to which slices and correspond segments have been removed by omission by way of a network device 50, for instance. Further, additionally or alternatively, external signalization as depicted by 58 in FIG. 1 may be used by decoder 38 or 52 to determine the decoding option or target to be used. The external signalization may already uniquely identify the coding option to be used or may merely exclude some coding options not to be used, or not being available due to, for instance, an

intermediate extraction process **14** in which case this external signalization may stem from the network device **50**, or may identify positively a set of coding options out of which the decoder shall select one on the basis of further information. Further, additionally or alternatively, side information in the inbound data stream such as explicit information on the yet decodable options may be used by decoder **38** or **52** to determine the decoding option or target to be used. Like external information, the latter information might have been included by the extracting entity, i.e. device **50**. Further, decoder may check the decoder capability levels associated with the decoding options as present in the parameter set **54**, to exclude some of the list of possible decoding option candidates when they conflict with the decoder's capability. A combination of some or all of these hints may form the decoding option indication, too. In case of any remaining ambiguity, the decoder may use the afore-mentioned priority to determine one of remaining possible decoding option candidates with highest priority.

[0282] Although some aspects have been described in the context of an apparatus, it is clear that these aspects also represent a description of the corresponding method, where a block or device corresponds to a method step or a feature of a method step. Analogously, aspects described in the context of a method step also represent a description of a corresponding block or item or feature of a corresponding apparatus. Some or all of the method steps may be executed by (or using) a hardware apparatus, like for example, a microprocessor, a programmable computer or an electronic circuit. In some embodiments, one or more of the most important method steps may be executed by such an apparatus.

[0283] The inventive data streams can be stored on a digital storage medium or can be transmitted on a transmission medium such as a wireless transmission medium or a wired transmission medium such as the Internet.

[0284] Depending on certain implementation requirements, embodiments of the invention can be implemented in hardware or in software. The implementation can be performed using a digital storage medium, for example a floppy disk, a DVD, a Blu-Ray, a CD, a ROM, a PROM, an EPROM, an EEPROM or a FLASH memory, having electronically readable control signals stored thereon, which cooperate (or are capable of cooperating) with a programmable computer system such that the respective method is performed. Therefore, the digital storage medium may be computer readable.

[0285] Some embodiments according to the invention comprise a data carrier having electronically readable control signals, which are capable of cooperating with a programmable computer system, such that one of the methods described herein is performed.

[0286] Generally, embodiments of the present invention can be implemented as a computer program product with a program code, the program code being operative for performing one of the methods when the computer program product runs on a computer. The program code may for example be stored on a machine readable carrier.

[0287] Other embodiments comprise the computer program for performing one of the methods described herein, stored on a machine readable carrier.

[0288] In other words, an embodiment of the inventive method is, therefore, a computer program having a program

code for performing one of the methods described herein, when the computer program runs on a computer.

[0289] A further embodiment of the inventive methods is, therefore, a data carrier (or a digital storage medium, or a computer-readable medium) comprising, recorded thereon, the computer program for performing one of the methods described herein. The data carrier, the digital storage medium or the recorded medium are typically tangible and/or non-transitional.

[0290] A further embodiment of the inventive method is, therefore, a data stream or a sequence of signals representing the computer program for performing one of the methods described herein. The data stream or the sequence of signals may for example be configured to be transferred via a data communication connection, for example via the Internet.

[0291] A further embodiment comprises a processing means, for example a computer, or a programmable logic device, configured to or adapted to perform one of the methods described herein.

[0292] A further embodiment comprises a computer having installed thereon the computer program for performing one of the methods described herein.

[0293] A further embodiment according to the invention comprises an apparatus or a system configured to transfer (for example, electronically or optically) a computer program for performing one of the methods described herein to a receiver. The receiver may, for example, be a computer, a mobile device, a memory device or the like. The apparatus or system may, for example, comprise a file server for transferring the computer program to the receiver.

[0294] In some embodiments, a programmable logic device (for example a field programmable gate array) may be used to perform some or all of the functionalities of the methods described herein. In some embodiments, a field programmable gate array may cooperate with a microprocessor in order to perform one of the methods described herein. Generally, the methods are advantageously performed by any hardware apparatus.

[0295] The apparatus described herein may be implemented using a hardware apparatus, or using a computer, or using a combination of a hardware apparatus and a computer.

[0296] The apparatus described herein, or any components of the apparatus described herein, may be implemented at least partially in hardware and/or in software.

[0297] The methods described herein may be performed using a hardware apparatus, or using a computer, or using a combination of a hardware apparatus and a computer.

[0298] The methods described herein, or any components of the apparatus described herein, may be performed at least partially by hardware and/or by software.

[0299] While this invention has been described in terms of several embodiments, there are alterations, permutations, and equivalents which fall within the scope of this invention. It should also be noted that there are many alternative ways of implementing the methods and compositions of the present invention. It is therefore intended that the following appended claims be interpreted as including all such alterations, permutations and equivalents as fall within the true spirit and scope of the present invention.

1. A method of decoding video comprising:

receiving, via a data stream, a picture that is partitioned into a plurality of segments that are composed of a plurality of slices;

decoding, from the data stream, a slice header that provides a start position corresponding to a respective slice of the plurality of slices relative to a segment onset position; and

determining an absolute address of the respective slice relative to a start of the picture, the determining including performing a summation corresponding to the start position and the segment onset position.

2. The method of claim 1, wherein a sub-picture is composed of a single segment of the plurality of segments.

3. The method of claim 1, wherein the slice header is included in a parameter set, and wherein the start position comprises a syntax element that indexes a segment in which the starting position is located, and wherein the syntax element comprises a variable length code.

4. A non-transitory processor readable medium having stored thereon instructions for causing at least one processor to perform the method of claim 1.

5. A video decoder comprising:

at least one processor configured to perform operations comprising:

receiving, via a data stream, a picture that is partitioned into a plurality of segments that are composed of a plurality of slices;

decoding, from the data stream, a slice header that provides a start position corresponding to a respective slice of the plurality of slices relative to a segment onset position; and

determining an absolute address of the respective slice relative to a start of the picture, the determining including performing a summation corresponding to the start position and the segment onset position.

6. The video decoder of claim 5, wherein a sub-picture is composed of a single segment of the plurality of segments.

7. The video decoder of claim 5, wherein the slice header is included in a parameter set, and wherein the start position comprises a syntax element that indexes a segment in which the starting position is located, and wherein the syntax element comprises a variable length code.

8. A method of encoding video comprising:

partitioning a picture into a plurality of segments that are composed of a plurality of slices;

providing a slice header that provides a start position corresponding to a respective slice of the plurality of slices relative to a segment onset position,

wherein an absolute address of the respective slice is determined relative to a start of the picture, the determining including performing a summation corresponding to the start position and the segment onset position; and

encoding the slice header and the respective slice into a data stream.

9. The method of claim 8, wherein the plurality of segments are traversed along a coding path sequentially, and wherein a segment of the plurality of segments is traversed completely before a next segment of the plurality of segments is traversed.

10. The method of claim 8, wherein the plurality of segments are coded without having coding interdependencies from each other.

11. The method of claim 8, wherein a sub-picture is composed of a single segment of the plurality of segments.

12. The method of claim 8, wherein the slice header is included in a parameter set, and wherein the start position comprises a syntax element that indexes a segment in which the starting position is located, and wherein the syntax element comprises a variable length code.

13. A non-transitory processor readable medium having stored thereon instructions for causing at least one processor to perform the method of claim 8.

14. A video encoder comprising:

at least one processor configured to perform operations comprising:

partitioning a picture into a plurality of segments that are composed of a plurality of slices;

providing a slice header that provides a start position corresponding to a respective slice of the plurality of slices relative to a segment onset position,

wherein an absolute address of the respective slice is determined relative to a start of the picture, the determining including performing a summation corresponding to the start position and the segment onset position; and

encoding the slice header and the respective slice into a data stream.

15. The video encoder of claim 14, wherein the plurality of segments are traversed along a coding path sequentially, and wherein a segment of the plurality of segments is traversed completely before a next segment of the plurality of segments is traversed.

16. The video encoder of claim 14, wherein the plurality of segments are coded without having coding interdependencies from each other.

17. The video encoder of claim 14, wherein a sub-picture is composed of a single segment of the plurality of segments.

18. The video encoder of claim 14, wherein the slice header is included in a parameter set, and wherein the start position comprises a syntax element that indexes a segment in which the starting position is located, and wherein the syntax element comprises a variable length code.

* * * * *